Resumen completo para el Parcial 1 — Programación 1

Este documento cubre **todo lo que se vio en las 16 clases hasta el 27 de abril** (el día antes del parcial), organizado para que sirva de estudio único de cara al parcial del **martes 28 de abril de 2026**.

Está pensado para alguien que **nunca programó antes**. Cada concepto se explica de cero, con palabras simples primero, después analogías de la vida real, y recién después el código. Cada término técnico la primera vez que aparece se explica.

Mapa de lo que vamos a ver

Parte	¿Qué se aprende?
0	Cómo es el parcial: formato, puntaje, qué esperar
1	Pseudocódigo y estructuras de control (clases 1–3)
2	JavaScript básico: variables, tipos, operadores, conversiones (clases 4–8)
3	Estructuras de control en JS: if/else, while, do-while, for (clases 4–10)
4	Strings: métodos, recorridos, inmutabilidad (clases 12–13, 15)
5	Funciones: definición, parámetros, return, invocación (clases 13–15)
6	Patrones que aparecen una y otra vez en parciales
7	Errores comunes y consejos directos del profesor
8	Checklist pre-parcial
9	Ejercicios resueltos del estilo del parcial
10	Preguntas de práctica

PARTE 0 — Cómo es el parcial

¿Qué te van a tomar?

El profesor (Wagner) y los profes de práctico (Oliver y Mauri) lo dijeron varias veces:

> "Van a hacer tres ejercicios y son tan buenos que el ejercicio más fácil tiene más puntos y el más difícil menos."

Traducción: El parcial es exactamente esta estructura. En la última clase antes del parcial (27/04), Wagner confirmó los puntajes:

Ejercicio	Puntaje	Nivel	Tipo
1	**7 puntos**	Básico	Programa entero en JS (sin función). Lee datos con `prompt`, valida, calcula contadores/máximo/mínimo/promedio, muestra con `console.log` o `alert`.
2	**5 puntos**	Medio	**Función** que recibe parámetros, hace algo, y retorna un valor. Se pide ejemplificar la invocación.
3	**3 puntos**	Avanzado	Programa o función más complejo (puede ser dibujar un patrón con doble for, o un algoritmo más rebuscado).
Total	**15 puntos**		

> "El de nivel más básico tiene más puntos que el de nivel avanzado." — el más fácil te da más puntos. **Empezá por el ejercicio 1**, asegurate los 7 puntos primero.

Reglas del parcial

> "Sin material, individual, dos horas, en papel, presencial."

- **Duración:** 2 horas
- **Sin material**, sin calculadora, sin celular (todo va al frente del salón)

- **No se puede ir al baño** durante las dos horas (vayan antes)
- Cada ejercicio se escribe **en una carilla diferente**
- Se entrega en hoja, no en computadora
- Se anotan en gestión antes
- **El parcial vale 15 puntos del total de 100** del curso. Sin mínimo de aprobación — se puede sacar 0 y todavía aprobar el curso si se hace bien el resto

Qué corrigen

> "Lo único que cambia es que se olviden de los punticomas, que no esté bien indentado el código, que tenga múltiples returns, cosas así."

- **Que funcione** (la lógica esté correcta)
- **Punto y coma al final de cada sentencia** (en JS no es obligatorio, pero se exige acá)
- **Indentación** (que el código tenga sangrías que muestran qué está adentro de qué — más adelante explico esto a fondo)
- **Un solo `return` por función** (regla del curso)
- **Nombres de variables mnemotécnicos** (no `a`, `b`, `x`; usar `cantidad`, `total`, `letraMaxima`)
- **Nombres en `camelCase`** (`letraMaxima`, no `letra_maxima` ni `LetraMaxima`)
- **Que se ejemplifique la invocación** de la función (cuando piden función)

Estrategia

> "Lean bien la letra. Si no entienden algo, pregunten."

> "Vayan por la solución simple. No se compliquen."

- Leer la letra **dos veces** antes de escribir nada

- Si algo es ambiguo, **preguntar** al docente
- En las hojas de borrador, **hacer corridas a mano** con ejemplos chicos (más adelante explico qué es una corrida a mano)
- **Empezar por el ejercicio que les sale primero** (no necesariamente en orden)
- **Solución simple > solución elegante** — primero que funcione

---

PARTE 1 — Pseudocódigo y estructuras de control

¿Qué es un programa? Lo más básico

Un **programa** es una lista de **instrucciones** que la computadora ejecuta una atrás de la otra. Es como una **receta de cocina**: pasos en orden.

Una **instrucción** es una orden: "sumá", "guardá", "mostrá", "preguntá". La computadora hace exactamente lo que le pedís — ni más ni menos.

¿Qué es un algoritmo?

> "Un algoritmo es una secuencia ordenada de pasos para resolver un tipo específico de problema."

Una receta es un algoritmo. Los pasos para hacer un sándwich son un algoritmo. Los pasos para resolver una ecuación son un algoritmo. Los pasos para encontrar el número más grande de una lista son un algoritmo.

Las cinco propiedades obligatorias de un algoritmo:

Propiedad	Qué quiere decir
Finito	Termina en algún momento. No es infinito.
Tiene entrada	Recibe datos para procesar (ej: dos números a sumar).
Tiene salida	Produce un resultado (ej: la suma de esos dos números).
Efectivo	Cada paso es algo concreto que se puede hacer.
Preciso	No deja lugar a interpretación. "Un golpe de horno" no es preciso. "180°C durante 25 minutos" sí.

¿Qué es pseudocódigo?

Pseudocódigo es escribir un algoritmo **en español**, con una estructura clara, **antes** de escribirlo en un lenguaje de programación real. Es un paso intermedio.

Sirve para pensar el problema sin preocuparse por la sintaxis exacta de ningún lenguaje. Una vez que el pseudocódigo está bien, traducirlo a código es lo más fácil.

Las tres estructuras de control

Con SOLO tres tipos de estructura se construye **cualquier** programa. No hay más. Acordate de esto, porque todo lo demás se reduce a esto:

1. **Secuencia** — pasos uno detrás del otro
2. **Decisión** — elegir entre opciones según una condición
3. **Iteración** — repetir algo varias veces

1. Secuencia

Pasos uno detrás del otro, en orden, sin condiciones.

```
prender el fuego
calentar la sartén
agregar manteca
```

Acá no hay nada que decidir, no hay nada que repetir. Simplemente se hacen las cosas en orden.

2. Decisión: SI / SI...OTRO CASO

Sirve para elegir entre dos caminos según si pasa algo o no.

> "El SI siempre va a preguntar una única vez la condición."

Una condición es algo que es **verdadero** o **falso**. Por ejemplo: "tengo frío" es verdadero o falso. "Llovió hoy" es verdadero o falso.

Forma simple: SI sin OTRO CASO

```
SI tengo frío
    ponerme el abrigo
```

Cómo se lee: "Si la condición 'tengo frío' es verdadera, ponerme el abrigo. Si es falsa, no hago nada y sigo."

Forma con OTRO CASO

```
SI tengo tiempo Y tengo mucha hambre
    pedir jugo, torta y waffles
OTRO CASO
    pedir solo café
```

Cómo se lee: "Si la condición es verdadera, pido el menú grande. Si es falsa, pido café."

Importante: Solo se ejecuta UN bloque, nunca los dos. O ejecutás el del SI, o ejecutás el del OTRO CASO.

La regla clave del SI

El SI **mira la condición una sola vez**. Si en ese momento es verdadera, hace el bloque y sigue. Si es falsa, salta el bloque y sigue. **No vuelve a preguntar más adelante**. Si querés que vuelva a preguntar, necesitás una iteración (ahora la veo).

3. Iteración: REPETIR X VECES

Sirve para repetir algo cuando ya sabés cuántas veces.

```
REPETIR 5 VECES
    tomar un pan
    poner queso
    poner jamón
```

Cómo se lee: "Hacer todo lo que está adentro 5 veces."

4. Iteración: MIENTRAS (la más importante para el parcial)

Sirve para repetir hasta que algo cambie.

```
MIENTRAS esté seca la tierra
    agregar 15 gotas de agua
```

Cómo funciona, paso a paso:

1. Pregunta: "¿La tierra está seca?"
2. Si la respuesta es SÍ, hace el bloque (agregar 15 gotas).
3. Vuelve al paso 1 y pregunta de nuevo.
4. En algún momento la respuesta va a ser NO, y ahí el MIENTRAS termina.

Detalle clave: Si la primera vez que pregunta la respuesta ya es NO, el bloque **nunca se ejecuta**. Por ejemplo, si la tierra ya estaba mojada cuando arrancó, no se agrega ni una gota.

LA REGLA DE ORO DEL MIENTRAS

> "Es muy importante que dentro de esa iteración modifique esa condición y en algún momento se vuelva falsa."

Si el código adentro del MIENTRAS no hace nada que cambie la condición, el bucle nunca termina. Eso se llama **loop infinito** y la computadora se queda colgada para siempre.

Ejemplo de loop infinito:

```
MIENTRAS la tierra esté seca
    cantar una canción          ← cantar no moja la tierra
```

La tierra nunca se va a mojar. El programa nunca termina.

Ejemplo correcto:

```
MIENTRAS la tierra esté seca
    agregar 15 gotas             ← esto eventualmente moja la tierra
```

5. Iteración: REPETIR...MIENTRAS (do-while)

Igual que el MIENTRAS, **pero garantiza que el bloque se ejecuta al menos una vez**, porque pregunta la condición al final.

```
REPETIR
    poner manteca en la sartén
    poner preparación
    esperar 30 segundos
    dar vuelta el panqueque
MIENTRAS haya preparación
```

Cómo funciona:

1. Hace el bloque (sin preguntar nada).
2. Después pregunta la condición.
3. Si la condición es verdadera, vuelve al paso 1.

4. Si es falsa, termina.

Cuándo usar cada iteración

Estructura	Cuándo usarla
`REPETIR X VECES`	Sabés exactamente cuántas veces (ejemplo: 10 números, 500 tornillos)
`MIENTRAS`	Repetís hasta que pase algo, y tiene sentido que pueda no ejecutarse nunca
`REPETIR ... MIENTRAS`	Repetís hasta que pase algo, pero el bloque **siempre** tiene que ejecutarse al menos una vez

Ejemplo: por qué la elección importa

Quiero hacer panqueques. Tengo dos versiones del algoritmo:

Versión A (con MIENTRAS):

```
MIENTRAS haya masa
    poner masa en la sartén
    cocinar
    sacar
```

Versión B (con REPETIR...MIENTRAS):

```
REPETIR
    poner masa en la sartén
    cocinar
    sacar
MIENTRAS haya masa
```

¿Qué pasa si arrancás sin masa preparada?

- **Versión A:** No hace nada (porque la primera vez ya no hay masa). Resultado: cero panqueques, nada se rompe.

- **Versión B:** Pone "nada" en la sartén, cocina "nada", saca "nada", y recién ahí pregunta. Resultado: un desastre.

Regla práctica: Si tiene sentido que el bloque pueda ejecutarse cero veces, usá MIENTRAS. Si por lógica del problema siempre tiene que ejecutarse al menos una vez, usá REPETIR...MIENTRAS.

Una garantía importante del MIENTRAS

> "Cuando termina el mientras, ¿en qué estado va a estar la condición? La única forma de que termine es que la condición me dé falso."

Cuando un MIENTRAS termina, sabés con certeza absoluta que la condición de salida se cumplió.

Si tu MIENTRAS dice `mientras semáforo NO esté en verde`, en el momento en que termina, **el semáforo está en verde, sí o sí**. No hace falta volver a verificarlo.

PARTE 2 — JavaScript básico

¿Qué es JavaScript?

JavaScript (o JS) es el lenguaje de programación que usás en este curso. Es el idioma en que la computadora va a recibir las instrucciones.

JavaScript es **interpretado** — quiere decir que la computadora lo lee y ejecuta línea por línea, sin un paso de "compilación" previo.

¿Qué es una variable?

Una **variable** es un **espacio en la memoria de la computadora donde guardás un dato** y al que le ponés un nombre para poder usarlo después.

Analogía: una variable es como un cajón con una etiqueta. La etiqueta es el nombre. Adentro del cajón hay una cosa (el dato).

```
[ edad ]   ← etiqueta (nombre de la variable)
   25      ← lo que hay adentro (el valor)
```

Cómo se crea una variable en JS

Para crear una variable se escribe la palabra `let`, después el nombre, después un `=`, después el valor inicial:

```
let edad = 25;
```

Cómo se lee: "Creá una variable llamada `edad` y poné adentro el valor `25`."

Cada parte:

- `let` — palabra obligatoria que dice "voy a crear una nueva variable"
- `edad` — nombre que vos le ponés
- `=` — el símbolo "asignar" (poner adentro)
- `25` — el valor inicial
- `;` — punto y coma al final, que indica "esta sentencia terminó"

Cambiar el valor de una variable

Una vez creada, se puede cambiar el valor sin volver a usar `let`:

```
let edad = 25;        // ahora edad vale 25
edad = 26;             // ahora edad vale 26
edad = edad + 1;       // ahora edad vale 27
```

Importante: una variable solo se "crea" una vez (con `let`). Después solo se le asigna un valor nuevo (sin `let`).

`let` vs `const` vs `var`

Palabra	Cuándo usarla
`let`	**Es lo que vamos a usar siempre** en este curso. Variable cuyo valor puede cambiar.
`const`	Constante. Su valor NO puede cambiar después de creada.
`var`	**NO usarla.** Es la forma vieja, quedó obsoleta.

> "El problema de usar `var` es que cuando definís una variable con `var`, el scope es todo el programa."

Scope = "alcance". Significa **dónde existe** una variable y dónde no. Las variables `let` solo existen en el bloque donde se crearon (un bloque es lo que está entre `{` y `}`). Las `var` se filtran a todos lados y eso causa errores.

Reglas para nombres de variables

- En **minúsculas**
- **Mnemotécnico** (que se entienda con leerlo): `puntajeMaximo`, no `pm`
- **Camel case** si tiene varias palabras: la primera palabra en minúscula, las siguientes con la primera letra en mayúscula, todo pegado. Ejemplo: `letraMaxima`, no `letra_maxima` ni `LetraMaxima`
- No abreviar: `cantidadEventos`, no `canEv`

Asignación: el operador `=`

El `=` en programación NO significa "es igual a" como en matemática. Significa **"poné lo de la derecha en lo de la izquierda"**.

```
let x = 5;          // poné 5 en x
x = x + 1;          // poné (lo que hay en x, más 1) en x. Ahora x vale 6.
```

Ese `x = x + 1` en matemática sería absurdo (¿cómo `x` puede ser igual a `x+1`?). En programación significa "tomá el valor actual de x, sumale 1, y guardalo de vuelta en x".

Atajos para asignar

```
let total = 0;
total = total + 5;    // forma larga
total += 5;            // atajo: hace lo mismo
total++;               // otro atajo: suma 1
```

Atajo	Equivale a
`x += 5`	`x = x + 5`
`x -= 5`	`x = x - 5`
`x *= 5`	`x = x * 5`
`x /= 5`	`x = x / 5`
`x++`	`x = x + 1`
`x--`	`x = x - 1`

Detalle raro: `i++` vs `++i`

Los dos suman 1 a la variable. La diferencia es **cuándo** se usa el valor:

```
let a = 3000;
console.log(a++);    // muestra 3000, DESPUÉS suma 1. Ahora a vale 3001.
console.log(++a);    // SUMA 1 primero, después muestra. a vale 3002,
muestra 3002.
```

Regla simple: Si el `++` está **después**, primero usa el valor y después suma. Si está **antes**, primero suma y después usa el valor.

En el parcial casi siempre vas a usar `i++` solo (en un for, por ejemplo), donde no importa el orden porque no estás usando el valor en ese momento.

¿Qué es un "tipo de dato"?

Un **tipo de dato** dice **qué clase de cosa** es un valor. ¿Es un número? ¿Es un texto? ¿Es un sí/no?

JavaScript tiene varios tipos. Estos son los que importan para el parcial:

Tipo	Qué representa	Ejemplos
`number`	Números (con o sin decimales)	`25`, `3.14`, `-7`, `0`
`string`	Texto (cadena de caracteres)	`"hola"`, `'que tal'`, `""`
`boolean`	Verdadero o falso	`true`, `false`
`undefined`	Una variable creada sin valor	`let x;` → x es undefined

Boolean: ¿qué es?

Un **boolean** (o booleano) es un valor que puede ser solo una de dos cosas: `true` (verdadero) o `false` (falso). Nada más.

Sirve para guardar respuestas a preguntas tipo sí/no:

- ¿Encontré lo que buscaba? → `true` o `false`
- ¿Ya terminé? → `true` o `false`
- ¿El número es par? → `true` o `false`

```
let encontrado = false;
let terminado = true;
let esPar = (n % 2 == 0);     // se asigna el resultado de la comparación
```

Cómo se sabe el tipo de una variable

Con la palabra `typeof`:

```
typeof 25          // "number"
typeof "hola"      // "string"
typeof false       // "boolean"
typeof undefined   // "undefined"
```

Cosa rara de JavaScript: una variable puede cambiar de tipo

```
let x = 23;        // x es number
x = "hola";        // ahora x es string. JS no se queja.
```

En la mayoría de los lenguajes esto sería error. En JS no. **Pero igual no lo hagas a propósito**, es confuso.

Strings: comillas simples o dobles

Para escribir un texto (string), usás comillas. Pueden ser dobles `"hola"` o simples `'hola'`. Las dos sirven, son equivalentes:

```
let texto1 = "hola";
let texto2 = 'hola';
// son lo mismo
```

Caracteres especiales dentro de strings

¿Qué pasa si querés meter una comilla dentro de un texto que ya está entre comillas? Usás la **contrabarra** (`\`) para "escapar" el carácter:

```
"Dijo \"hola\""    // el texto: Dijo "hola"
'Dijo \'hola\''    // el texto: Dijo 'hola'
```

Otros caracteres especiales:

Secuencia	Qué hace
`\n`	Salto de línea (siguiente línea)
`\t`	Tabulación (espaciado grande)
`\\`	Una contrabarra normal
`\"`	Una comilla doble
`\'`	Una comilla simple

```
console.log("Hola\nMundo");
// Imprime:
// Hola
// Mundo
```

Acceder al contenido de un string: `length` y `charAt(i)`

Antes de seguir necesitás conocer dos cosas básicas de los strings, porque las voy a usar en ejemplos más adelante. (En PARTE 4 las explico en detalle junto con todos los demás métodos de strings.)

`texto.length` — cuántos caracteres tiene

```
"hola".length     // 4   (la palabra "hola" tiene 4 caracteres)
"".length         // 0   (string vacío)
```

OJO: se escribe **sin paréntesis**. `length` es una **propiedad**, no una función.

`texto.charAt(i)` — el carácter en la posición `i`

Las posiciones de un string **arrancan en 0**, no en 1.

```
texto = "hola"

posición:  0  1  2  3
carácter:  h  o  l  a
```

```
"hola".charAt(0)    // "h"   (primer carácter, posición 0)
"hola".charAt(1)    // "o"
"hola".charAt(3)    // "a"   (último carácter)
"hola".charAt(99)   // ""    (posición que no existe → string vacío, no
rompe)
```

OJO: se escribe **con paréntesis** y un número adentro. `charAt` es una **función**.

Regla clave: la última posición válida de un string es `length - 1`. Si `texto.length` es 4, las posiciones válidas son 0, 1, 2, 3 — la posición 4 ya no existe.

Operadores aritméticos (los matemáticos)

Un **operador** es un símbolo que **opera** sobre uno o varios valores. Los aritméticos hacen cuentas:

Operador	Hace
`+`	Suma (o concatena strings — cuidado, lo veo abajo)
`-`	Resta
`*`	Multiplica
`/`	Divide
`%`	**Módulo (resto de la división)** — clave para el parcial
`**`	Potencia (también `Math.pow(a, b)`)

El módulo (`%`) en detalle — MUY IMPORTANTE

Aparece en el 80% de los parciales. Si lo entendés bien, te ahorrás muchísimo.

La idea, sin matemática

Olvidate de "módulo" por un segundo. Pensá en esto:

> Tenés **22 caramelos** y los repartís entre **5 amigos** en partes iguales. A cada uno le tocan **4 caramelos** (porque 4 × 5 = 20). ¿Cuántos te sobran? **2 caramelos** (porque 22 − 20 = 2, y no alcanzan para darle uno más a cada uno).

Eso que **te sobra** es exactamente lo que el operador `%` te devuelve.

```
22 % 5    // 2   ← lo que sobra cuando repartís 22 entre 5
```

Más ejemplos:

```
15 % 3   // 0   (3 entra justo 5 veces en 15, no sobra nada)
10 % 3   // 1   (3 entra 3 veces — son 9 — sobra 1)
10 % 2   // 0   (2 entra 5 veces en 10, sobra 0)
7 % 4    // 3   (4 entra 1 vez, sobran 3)
3 % 10   // 3   (10 no entra ni una vez en 3, sobran los 3)
9 % 9    // 0   (9 entra 1 vez justo, sobra 0)
```

La regla en una frase: `a % b` es lo que te sobra después de dividir `a` entre `b` la mayor cantidad de veces posible.

Por qué los "trucos" del módulo funcionan

Truco 1 — Saber si un número es par: `n % 2 == 0`

Si dividís entre 2 y no te sobra nada, es porque entró justo. Esos son los pares: 2, 4, 6, 8, 10...

- `4 % 2` → 0 → par
- `7 % 2` → 1 → impar (sobra uno)

Truco 2 — Saber si es múltiplo de 3: `n % 3 == 0`

Si dividís entre 3 y no sobra nada, es múltiplo. Los múltiplos de 3 son 3, 6, 9, 12...

- `12 % 3` → 0 → es múltiplo
- `13 % 3` → 1 → no, sobra uno

Truco 3 — Saber si `a` es divisible entre `b`: `a % b == 0`

Versión general. Si entra justo, no sobra nada.

Truco 4 — Quedarse con el último dígito: `n % 10`

Si tenés 347 y lo dividís entre 10, te entran 34 veces (340) y sobran **7**. 7 es el último dígito.

- `347 % 10` → 7
- `1280 % 10` → 0
- `25 % 10` → 5

Tabla de uso rápido

Para saber si...	Hacés	Por qué
Es par	`n % 2 == 0`	Si se divide justo entre 2, no sobra
Es impar	`n % 2 ≠ 0`	Si sobra 1 al dividir entre 2
Es múltiplo de 3	`n % 3 == 0`	Si entra justo 3
Es múltiplo de 5	`n % 5 == 0`	Si entra justo 5
Es divisible entre `b`	`a % b == 0`	Si entra justo `b`
Sacar el último dígito	`n % 10`	El sobrante de dividir entre 10

La sobrecarga del `+` (la trampa más común)

El `+` tiene DOS comportamientos diferentes en JavaScript:

1. **Si los dos lados son números**, suma como en matemática.

2. **Si al menos uno de los dos es un string (texto)**, NO suma — los **pega uno al lado del otro** (eso se llama **concatenar**).

```
3 + 4          // 7    (suma, los dos son números)
"3" + 4        // "34" (concatena, porque "3" es string)
3 + "4"        // "34" (igual, no importa el lado)
```

¿Qué pasa cuando hay varios `+`?

Se evalúan **de izquierda a derecha**. Cada `+` mira a sus dos lados en ese momento.

```
3 + 4 + "5"
```

Paso a paso:

1. Primero hace `3 + 4`. Los dos son números. Suma. Resultado intermedio: `7`.
2. Después tiene `7 + "5"`. Uno es string. Concatena. Resultado final: `"75"`.

```
"3" + 4 + 5
```

Paso a paso:

1. Primero hace `"3" + 4`. Uno es string. Concatena. Resultado intermedio: `"34"`.
2. Después tiene `"34" + 5`. Concatena. Resultado final: `"345"`.

¿Por qué importa esto en el parcial?

Porque cuando pedís un dato con `prompt( ... )`, **siempre te devuelve un string**. Si no lo convertís a número, las "sumas" van a concatenar:

```
let edad = prompt("Edad");         // edad es "25" (string)
console.log(edad + 5);              // "255" ← BUG
console.log(parseInt(edad) + 5);    // 30   ← bien
```

(Más adelante explico `parseInt`.)

Operadores de comparación

Los operadores de **comparación** comparan dos valores y devuelven `true` o `false`.

Operador	Significado
`==`	Igual (compara solo valor)
`===`	**Estrictamente igual** (compara valor Y tipo)
`≠`	Distinto (compara solo valor)
`≠=`	Estrictamente distinto
`<`	Menor
`>`	Mayor
`≤`	Menor o igual
`≥`	Mayor o igual

`==` vs `===`: la diferencia

`==` solo mira si el **valor** es el mismo (y convierte tipos automáticamente para comparar).
`===` mira si el **valor** Y el **tipo** son iguales.

```
5 == "5"     // true   (mismo valor, distinto tipo, pero == lo deja pasar)
5 === "5"    // false  (distinto tipo, === es estricto)
5 === 5      // true   (mismo valor y tipo)
```

En el parcial: Da igual usar `==` o `===` cuando estás comparando dos cosas que sabés que son del mismo tipo. Si tenés dudas, usá `===`.

Cuidado con `=` vs `==`

Símbolo	Qué hace
`=`	**Asigna** (pone un valor en una variable). NO compara.
`==`	**Compara**. Devuelve true o false.

Error clásico:

```
if (x = 5) {  ...  }    // ESTO ASIGNA 5 a x. La condición casi siempre es true.
if (x == 5) {  ...  }   // bien, compara
```

Operadores lógicos

El problema que resuelven

A veces necesitás **combinar varias condiciones** para tomar una decisión, no una sola.

Ejemplos de la vida real:

- "Si tengo dinero **Y** está abierto el local, voy a comprar." → necesito que se cumplan **dos cosas a la vez**.
- "Si llueve **O** hace frío, llevo paraguas." → me alcanza con que se cumpla **al menos una**.
- "Voy al parque si **NO** llueve." → necesito que **no se cumpla** algo.

Los **operadores lógicos** son los símbolos que te dejan combinar condiciones así. Hay tres:

Operador	Nombre	Significado
`&&`	AND (y)	Devuelve `true` solo si **ambos** lados son `true`
`||`	OR (o)	Devuelve `true` si **al menos un** lado es `true`
`!`	NOT (no, negado)	Invierte: convierte `true` en `false` y viceversa

Los operadores lógicos siempre devuelven un valor booleano (`true` o `false`).

AND (`&&`) — el "y"

Devuelve `true` **solo si los dos lados son** `true`. En cualquier otro caso, devuelve `false`.

Ejemplo concreto:

```
let edad = 20;
let tienePase = true;

if (edad ≥ 18 && tienePase) {
    alert("Puede entrar");
}
```

Cómo se lee: "Si la edad es mayor o igual a 18 **Y** tiene pase, mostrar 'Puede entrar'."

Solo entra al `if` si **las dos cosas** se cumplen:

- Si la edad es menor a 18 → no entra.
- Si no tiene pase → no entra.
- Si las dos fallan → tampoco entra.
- Solo entra cuando **ambas** son verdaderas.

Todos los casos posibles del AND (la "tabla de verdad")

Como cada condición puede ser `true` o `false`, y `&&` recibe dos condiciones, hay **4 combinaciones posibles**. Las podemos listar todas en una **tabla de verdad** — una tablita que muestra qué resultado da el operador en cada caso.

Cómo leer la tabla: `A` y `B` son nombres genéricos para "una condición cualquiera" y "otra condición cualquiera". Cada una puede valer `true` o `false`. La columna `A && B` es el resultado del AND para esa combinación.

A	B	A && B
`false`	`false`	`false`
`false`	`true`	`false`
`true`	`false`	`false`
`true`	`true`	**`true`**

Cómo se interpreta: Solo la **última fila** (cuando A y B son los dos `true`) da `true`. Todas las demás filas dan `false`.

Aplicado al ejemplo de arriba: `edad ≥ 18` da `true`, `tienePase` da `true`. Las dos verdaderas → última fila → resultado `true` → entra al `if`.

> 💡 En libros de matemática a veces verás las tablas de verdad escritas con **V** (verdadero) y **F** (falso) en lugar de `true` / `false`. Significan lo mismo — son dos formas de escribir lo mismo.

OR (`||`) — el "o"

Devuelve `true` si **al menos uno** de los dos lados es `true`. Solo devuelve `false` cuando los dos son `false`.

Ejemplo concreto:

```
let dia = "sábado";

if (dia == "sábado" || dia == "domingo") {
    alert("Es fin de semana");
}
```

Cómo se lee: "Si el día es 'sábado' **O** es 'domingo', mostrar 'Es fin de semana'."

Con que se cumpla **una** de las dos, alcanza para entrar al `if`.

Todos los casos posibles del OR

A	B	A || B
`false`	`false`	`false`
`false`	`true`	**`true`**
`true`	`false`	**`true`**
`true`	`true`	**`true`**

Cómo se interpreta: Solo la **primera fila** (cuando los dos son `false`) da `false`. Todas las otras dan `true`.

NOT (`!`) — el "no" / negado

A diferencia de AND y OR, NOT recibe **un solo valor** (no dos) y lo invierte:

- Si era `true`, devuelve `false`.
- Si era `false`, devuelve `true`.

Ejemplo concreto:

```
let llueve = false;

if (!llueve) {
    alert("Vamos al parque");
}
```

Cómo se lee: "Si **NO** llueve, vamos al parque."

Paso a paso:

1. `llueve` vale `false`.
2. `!llueve` invierte ese valor → da `true`.
3. La condición del `if` es `true` → entra al bloque → muestra "Vamos al parque".

Todos los casos posibles del NOT

Como recibe un solo valor, solo hay 2 combinaciones posibles:

A	!A
`true`	`false`
`false`	`true`

Simplemente da el contrario. Nada más.

Circuito corto — qué es y por qué importa

¿Qué es "evaluar"?

Una **expresión** es cualquier "cosa que tiene un valor": un número (`5`), una variable (`x`), una cuenta (`2 + 3`), una comparación (`5 > 3`), una función con resultado (`Math.abs(-7)`).

Evaluar una expresión = averiguar cuánto vale. Cuando JS ve `2 + 3`, lo evalúa y obtiene `5`. Cuando ve `5 > 3`, lo evalúa y obtiene `true`.

¿Qué es el circuito corto?

Cuando JS ve una operación con `&&` o `||`, **no siempre evalúa los dos lados**. A veces ya sabe el resultado con solo mirar el primero, y se ahorra el segundo.

Caso del OR (`||`): si el primero es true, no mira el segundo

¿Por qué? Porque OR da true si **al menos uno** es true. Si el primero ya es true, el resultado va a ser true sí o sí. Da lo mismo qué sea el segundo.

```
true || cualquierCosa()    // ni mira "cualquierCosa()"
```

JS dice: "Bueno, el primero es true, ya sé que el resultado es true, no necesito mirar más." Y no ejecuta `cualquierCosa()`.

Caso del AND (`&&`): si el primero es false, no mira el segundo

¿Por qué? Porque AND da true solo si **los dos** son true. Si el primero ya es false, el resultado va a ser false sí o sí.

```
false && cualquierCosa()   // ni mira "cualquierCosa()"
```

¿Por qué esto importa en el parcial?

Te permite poner una **condición de seguridad** primero, para que la segunda condición no se ejecute si va a romper algo.

Ejemplo concreto

> Recordá: vimos en la sección anterior que `texto.length` es la cantidad de caracteres del string, y `texto.charAt(i)` es el carácter en la posición `i`. Las posiciones arrancan en 0.

Imaginate que tenés un texto `"hola"` (4 caracteres, posiciones 0, 1, 2, 3) y querés ver si la posición 10 tiene la letra "a". **Pero la posición 10 ni siquiera existe** en ese texto.

```
let texto = "hola";
let i = 10;

// MAL: esto puede romperse si i es muy grande
if (texto.charAt(i) == "a") { ... }

// BIEN: primero chequeo que i sea una posición válida
if (i < texto.length && texto.charAt(i) == "a") { ... }
```

¿Qué dice la condición de seguridad `i < texto.length`?

Significa: "la posición `i` es menor que la cantidad total de caracteres". Como las posiciones válidas son 0 hasta `length - 1`, esta condición es verdadera solo cuando `i` es una posición que realmente existe en el texto.

Cómo funciona la versión BIEN, paso a paso:

1. JS evalúa la primera condición: `i < texto.length`.
2. `i` vale 10. `texto.length` vale 4 (la palabra "hola" tiene 4 caracteres).
3. JS calcula `10 < 4`. Esto es **false**.
4. Como el operador es `&&` y la primera condición ya dio false, **JS no evalúa la segunda parte** (esto es el circuito corto). Salta `texto.charAt(i) == "a"` por completo.
5. El resultado de todo el `if` es `false`. No entra al bloque.

¿Y si no hubiéramos puesto el chequeo de seguridad?

JS habría intentado hacer `texto.charAt(10)` directamente. Como la posición 10 no existe en `"hola"`, te devuelve un string vacío `""`. La comparación `"" == "a"` da false (que en este caso parece "funcionar"), pero en otros contextos te puede dar valores inesperados que rompen tu lógica más adelante. **Mejor evitar el problema desde el inicio**.

Regla práctica: Poné las condiciones de seguridad **a la izquierda** del `&&`. La condición de la izquierda actúa como "filtro" — si falla, JS ni intenta evaluar la de la derecha.

Variables booleanas

¿Qué es una variable booleana?

Una **variable booleana** es una variable que guarda `true` o `false`. Nada más.

```
let encontrado = false;
let esValido = true;
let hubo = false;
```

Las usás para "marcar" si pasó algo. Por ejemplo, recorrer una lista buscando un valor: arrancás con `encontrado = false`, y si lo encontrás, lo cambiás a `true`.

> Hay una **regla del profe** sobre cómo escribir `if` con variables booleanas (forma corta vs forma larga). La vemos en PARTE 3, cuando expliquemos `if/else` en JavaScript. Y hay otra **regla similar para** `return` que vemos en PARTE 5, cuando expliquemos funciones.

Conversiones entre tipos

A veces necesitás convertir un dato de un tipo a otro. Por ejemplo, el `prompt` te devuelve un string, pero querés un número para hacer cuentas.

De string a número

Función	Qué hace
`parseInt("123")`	Devuelve `123` (entero, sin decimales). Si encuentra una letra, corta ahí.
`parseFloat("12.5")`	Devuelve `12.5` (con decimales).
`Number("123")`	Devuelve `123`. Más estricto: si tiene cualquier letra, devuelve `NaN`.

```
parseInt("123abc")     // 123 (ignora "abc")
parseInt("12.7")       // 12  (corta en el punto, da entero)
parseFloat("12.7")     // 12.7
Number("123abc")       // NaN
parseInt("abc123")     // NaN (empieza con letra)
```

Truco poderoso: `parseInt` saca los ceros a la izquierda

Si tenés un string que representa un número con ceros adelante (por ejemplo `"00001234"`), `parseInt` te devuelve el número limpio sin esos ceros:

```
parseInt("00001234")     // 1234
parseInt("00000100")     // 100
parseInt("000")          // 0
```

Por qué importa: Apareció exactamente este caso en un parcial real (clase 16): "la contraseña es CA seguido de 18 dígitos que arrancan con ceros y terminan con el número de socio". En vez de recorrer con un for buscando dónde terminan los ceros, una sola línea lo resuelve:

```
let numeroSocio = parseInt(contrasena.slice(2));    // saca "CA", convierte
a número
```

Si la contraseña era `"CA0000000000000001234"`, `numeroSocio` queda en `1234`. Sin for, sin recorridos, sin variables booleanas. **Es el ejemplo perfecto del consejo del profe**: "no siempre necesitás recorrer; a veces hay una solución mucho más sencilla con las funciones que ya tenés".

¿Qué es `NaN`?

`NaN` = "Not a Number" (no es un número). Es lo que devuelven las operaciones que no tienen sentido como número:

- `parseInt("hola")` → `NaN`
- `0 / 0` → `NaN`
- `"abc" * 2` → `NaN`

De número a string

```
let x = 25;
x.toString()          // "25"
"" + x                // "25" (truco: sumar string vacío convierte)
```

Redondear decimales para mostrar

```
let n = 9.5467;
n.toFixed(2)          // "9.55" (cuidado: devuelve string, no número)
```

Constantes especiales del parcial

JavaScript tiene unos **valores predefinidos** que conviene conocer porque aparecen en los parciales:

Constante	Qué es
`Number.MAX_SAFE_INTEGER`	El **entero positivo más grande** que JS puede representar con seguridad
`Number.MIN_SAFE_INTEGER`	El **entero negativo más chico** (más negativo)
`Number.MAX_VALUE`	El número **con decimales más grande**
`Number.MIN_VALUE`	El número positivo **más cercano a 0** (NO es negativo, es un decimal chiquitito como 0.000...001)
`Infinity`	"Infinito". Resultado de `1 / 0`
`-Infinity`	"Menos infinito". Resultado de `-1 / 0`
`NaN`	"Not a Number". Resultado de `0 / 0` o `parseInt("abc")`
`Math.PI`	El número π = 3.14159...

¿Para qué sirve `MAX_SAFE_INTEGER` y `MIN_SAFE_INTEGER` en el parcial?

Para **inicializar variables** que van a buscar el máximo o mínimo.

Cuando buscás el máximo, arrancás `max` en el valor más chico posible (`MIN_SAFE_INTEGER`), así cualquier número real va a ser mayor y se actualiza.

Cuando buscás el mínimo, arrancás `min` en el valor más grande posible (`MAX_SAFE_INTEGER`), así cualquier número real va a ser menor y se actualiza.

(Más adelante en la sección de patrones doy ejemplos completos.)

Casos raros que conviene conocer

```
1 / 0            // Infinity
-1 / 0           // -Infinity
0 / 0            // NaN
parseInt("abc") // NaN
"3" * "4"        // 12  (JS convierte automáticamente al multiplicar)
"3" - 1          // 2   (JS convierte al restar)
"3" + 1          // "31" (concatena, no suma)
```

Cómo pedir y mostrar datos

`prompt( ... )` — pedir al usuario

`prompt` muestra una ventanita donde el usuario escribe algo y le da OK. Lo que escribe te lo devuelve como string.

```
let nombre = prompt("Ingrese su nombre");
let edad = parseInt(prompt("Ingrese su edad"));
```

REGLA CRÍTICA: `prompt` **siempre** devuelve string. Si vas a hacer cuentas con el dato, parsealo a número.

`alert( ... )` — mostrar un popup

`alert` muestra una ventanita con un mensaje y un botón OK.

```
alert("Hola " + nombre);
alert("Tu edad es " + edad);
```

`console.log( ... )` — escribir en la consola

`console.log` escribe un mensaje en la **consola del navegador** (la pantalla de mensajes que ven los desarrolladores).

```
console.log("Hola " + nombre);
console.log("La suma es " + suma);
```

¿Cuál uso?

- Si la letra dice **"mostrar por consola"** → `console.log`
- Si la letra dice **"mostrar con alert"** → `alert`
- Si no dice → cualquiera. El profe prefiere `console.log` para programas largos porque no te interrumpe.

PARTE 3 — Estructuras de control en JavaScript

Ahora vamos a ver cómo se escriben en JS las estructuras que vimos en pseudocódigo.

¿Qué es un "bloque de código"?

Un **bloque** es un grupo de instrucciones que están entre **llaves** `{` y `}`. Las estructuras de control en JS usan bloques para indicar qué se ejecuta en cada caso.

```
{
    let x = 5;
    let y = 10;
    console.log(x + y);
}
```

Las llaves marcan el principio y el final del bloque.

¿Qué es la indentación?

Indentación = la sangría (los espacios al principio de cada línea) que se usa para mostrar visualmente qué está adentro de qué.

```
if (edad ≥ 18) {
    alert("Mayor de edad");          ← este alert está adentro del if (4
espacios de sangría)
    alert("Puede votar");            ← este también
}
alert("Esto se muestra siempre");    ← este NO está adentro del if (sin
sangría)
```

En JS la indentación **no es obligatoria** (el código funciona igual sin ella), pero se exige en el parcial porque hace el código legible.

Convención: 4 espacios (o un tab) por cada nivel de "adentro de".

`if / else if / else`

Es la versión JS del SI / OTRO CASO en pseudocódigo.

Estructura básica

```
if (condicion) {
    // bloque que se ejecuta si la condición es verdadera
}
```

Cómo se lee: "Si la condición es verdadera, ejecutar el bloque. Si es falsa, saltearlo."

Con `else`

```
if (condicion) {
    // se ejecuta si verdadera
} else {
    // se ejecuta si falsa
}
```

Cómo se lee: "Si verdadera, hacé esto. Si no, hacé lo otro."

Solo se ejecuta UN bloque, nunca los dos.

Con `else if`

Si tenés más de dos casos, podés encadenar con `else if`:

```
if (nota ≥ 86) {
    alert("Exonerado");
} else if (nota ≥ 70) {
    alert("Aprobado, va a examen");
} else {
    alert("Perdió el curso");
}
```

Cómo se lee, paso a paso:

1. Si `nota ≥ 86`, mostrar "Exonerado" y terminar.
2. Si no, **probar la siguiente**: ¿`nota ≥ 70`? Si sí, mostrar "Aprobado" y terminar.
3. Si no se cumplió ninguna, hacer el `else` final: "Perdió el curso".

Detalle clave: Las condiciones se prueban en orden. Apenas una se cumple, se ejecuta su bloque y el resto se saltea, aunque también pudieran cumplirse.

`else` es opcional

Podés tener un `if` solo, sin `else`:

```
if (edad ≥ 18) {
    alert("Mayor");
}
// si edad < 18, simplemente no pasa nada
```

`if` anidado vs `else if`

Anidado = un `if` adentro de otro:

```
if (edad ≥ 18) {
    if (tieneCarnet) {
        alert("Puede manejar");
    } else {
        alert("Mayor pero sin carnet");
    }
}
```

Encadenado con `else if`:

```
if (nota ≥ 86) {
    alert("Exonerado");
} else if (nota ≥ 70) {
    alert("Aprobado");
} else {
    alert("Perdió");
}
```

Los dos son válidos. Usás anidado cuando la segunda condición depende de que la primera se haya cumplido. Usás encadenado cuando son **alternativas mutuamente excluyentes**.

Truco: `if` separados vs `if/else`

Si las condiciones son **mutuamente excluyentes** (no pueden cumplirse las dos a la vez), podés usar `if` separados sin problemas:

```
if (antiguedad ≥ 5 && antiguedad ≤ 10) {
    sueldo = sueldo * 1.10;
}
if (antiguedad > 10) {
    sueldo = sueldo * 1.20;
}
```

> "Como de estas dos, nunca van a entrar en ambas, lo puedo tener como dos `if` separados."

Pero **si pueden solaparse**, tenés que usar `else if` para evitar entrar dos veces.

Regla del profe: `if` con variables booleanas

> "Esta forma de escribir no la voy a usar más: `if (paso == true)`. Esto es lo mismo que poner solo `if (paso)`."

(Recordá lo que vimos en PARTE 2: una variable booleana es la que guarda `true` o `false`.)

Por qué `if (paso == true)` es lo mismo que `if (paso)`:

- Si `paso` ya vale `true`, entonces `paso == true` también es `true`. Las dos formas entran al `if`.
- Si `paso` vale `false`, entonces `paso == true` también es `false`. Ninguna entra.

Comparar una variable booleana con `true` es **redundante** — la variable misma ya tiene el valor true/false que necesita el `if`.

Mal (innecesariamente largo):

```
if (terminado == true) { ... }
if (encontrado == false) { ... }
```

Bien (la forma corta):

```
if (terminado) { ... }
if (!encontrado) { ... }
```

Tabla de equivalencias:

Forma larga	Forma corta
`if (x == true)`	`if (x)`
`if (x == false)`	`if (!x)`

> Hay una regla equivalente para `return` en funciones — la veremos en PARTE 5 cuando expliquemos funciones.

`while` (mientras)

Es la versión JS del MIENTRAS en pseudocódigo.

Estructura

```
while (condicion) {
    // se repite mientras la condición sea verdadera
}
```

Cómo se lee:

1. Pregunta: ¿la condición es verdadera?
2. Si SÍ, ejecuta el bloque.
3. Vuelve al paso 1.

4. Si NO, termina.

Ejemplo concreto: contar de 1 a 5

```
let i = 1;
while (i ≤ 5) {
    console.log(i);
    i++;
}
```

Corrida a mano (qué pasa paso a paso):

Paso	i	¿i <= 5?	Acción
1	1	Sí	imprime 1, después i++ deja i en 2
2	2	Sí	imprime 2, i pasa a 3
3	3	Sí	imprime 3, i pasa a 4
4	4	Sí	imprime 4, i pasa a 5
5	5	Sí	imprime 5, i pasa a 6
6	6	**No**	Termina

Resultado: imprime 1, 2, 3, 4, 5.

Trampa clásica: olvidarse de modificar la variable

```
let i = 1;
while (i ≤ 5) {
    console.log(i);
    // ¡FALTA i++ !
}
```

Este `while` nunca termina. `i` siempre vale 1, la condición siempre es true, y se queda colgado. Eso es **loop infinito**.

Patrón: validar entrada

```
let medida = parseInt(prompt("Medida"));
while (medida ≤ 0) {
    alert("Inválido, reingrese");
    medida = parseInt(prompt("Medida"));
}
// acá medida es > 0, garantizado
```

Cómo funciona, paso a paso: El usuario ingresa algo. Si es positivo, el `while` no entra y sigue. Si es negativo o cero, entra al `while`, le avisa, y le vuelve a pedir. Repite hasta que ingrese algo positivo.

Patrón: leer hasta valor centinela

Un **valor centinela** es un valor especial que significa "terminé de ingresar datos". Por ejemplo, "ingrese números, 0 para terminar".

```
let dato = parseInt(prompt("Ingrese dato (0 para terminar)"));
while (dato ≠ 0) {
    // procesar dato
    dato = parseInt(prompt("Ingrese dato (0 para terminar)"));
}
```

Patrón: bandera booleana para cortar antes

Una **bandera** es una variable booleana que marca si pasó algo:

```
let i = 0;
let encontrado = false;
while (i < texto.length && !encontrado) {
    if (texto.charAt(i) == "a") {
        encontrado = true;
    }
    i++;
}
```

Cómo funciona:

- Mientras quede texto por revisar Y no haya encontrado nada, sigue.
- Apenas encuentra una "a", `encontrado` se vuelve true.
- En la siguiente iteración, `!encontrado` es false, y el while corta.

`do ... while` (repetir-mientras)

Es la versión JS del REPETIR...MIENTRAS en pseudocódigo.

Estructura

```
do {
    // se ejecuta al menos una vez
} while (condicion);
```

Diferencia con `while`: `do-while` evalúa la condición **al final**, así que el bloque siempre se ejecuta **al menos una vez**.

Ejemplo: forzar al usuario a ingresar S o N

```
let respuesta;
do {
    respuesta = prompt("¿Continuar? (s/n)");
} while (respuesta ≠ "s" && respuesta ≠ "n");
```

Cómo funciona, paso a paso: Pide la respuesta sin preguntar nada antes. Después chequea: si no es "s" ni "n", vuelve a pedir.

`for` — la estructura más usada en el parcial

El `for` es una forma compacta de escribir un `while` cuando ya sabés cuántas veces vas a iterar.

Estructura

```
for (inicializacion; condicion; incremento) {
    // bloque
}
```

Tiene **tres partes** entre paréntesis, separadas por punto y coma:

1. **Inicialización**: lo que se hace al principio (típicamente crear una variable contadora).
2. **Condición**: se chequea ANTES de cada vuelta. Si es false, termina.
3. **Incremento**: se ejecuta al final de cada vuelta.

Ejemplo básico: contar de 1 a 10

```
for (let i = 1; i ≤ 10; i++) {
    console.log(i);
}
```

Cómo se lee:

- "Crea `i` con valor 1."
- "Mientras `i ≤ 10`, ejecuta el bloque."
- "Después de cada vuelta, hace `i++` (sumar 1 a i)."

Cómo funciona el `for` paso a paso

1. Ejecuta la **inicialización** (`let i = 1`) — UNA SOLA VEZ al principio.
2. Evalúa la **condición** (`i ≤ 10`).
3. Si es **true**, ejecuta el **bloque** (lo que está entre `{` y `}`).
4. Ejecuta el **incremento** (`i++`).
5. Vuelve al paso 2.
6. Si es **false**, termina.

Corrida a mano completa de un for chiquito

```
for (let i = 1; i ≤ 3; i++) {
    console.log("Vuelta " + i);
}
```

Paso	Qué pasa	i	Imprime
1	Crea i=1	1	—
2	¿1 <= 3? Sí	1	"Vuelta 1"
3	i++	2	—
4	¿2 <= 3? Sí	2	"Vuelta 2"
5	i++	3	—
6	¿3 <= 3? Sí	3	"Vuelta 3"
7	i++	4	—
8	¿4 <= 3? **No**, termina	4	—

Patrón: recorrer N veces

```
for (let i = 1; i ≤ n; i++) {
    // se ejecuta n veces
}
```

O empezando desde 0:

```
for (let i = 0; i < n; i++) {
    // se ejecuta n veces (de i=0 a i=n-1)
}
```

Las dos formas son equivalentes en cantidad de iteraciones.

Patrón: recorrer un string

Para leer un string letra por letra:

```
for (let i = 0; i < texto.length; i++) {
    let letra = texto.charAt(i);
    // hacer algo con letra
}
```

Por qué `i = 0` y `i < texto.length`: las posiciones de un string van de 0 a `length - 1`. Si ponés `i ≤ texto.length`, te vas a una posición que no existe.

Patrón: doble for (anidado)

Un `for` adentro de otro `for`:

```
for (let i = 0; i < n; i++) {
    for (let j = 0; j < m; j++) {
        // se ejecuta n*m veces
    }
}
```

Cómo funciona: por **cada** iteración del for exterior, el for interior se ejecuta **completo**. Si el exterior va de 0 a 2 (3 vueltas) y el interior de 0 a 3 (4 vueltas), el bloque interior se ejecuta 3 × 4 = 12 veces.

Cuándo se usa: cuando hay que dibujar una figura con filas y columnas, o cuando por cada elemento hay que recorrer otra cosa entera.

Saltar de a 2 (o más)

```
for (let i = 0; i < n; i += 2) {
    // procesa solo posiciones pares: 0, 2, 4, 6, ...
}
```

Los 5 patrones clásicos que aparecen en TODO parcial

¿Qué es un "patrón"?

Un **patrón** es una **forma típica de resolver un tipo de problema** que se repite muchas veces. No es una regla obligatoria — es una "receta probada" que aprendés una vez y reutilizás siempre que aparezca un problema parecido.

Analogía: los patrones son como las jugadas básicas en un deporte. Una vez que las sabés, las reconocés en cualquier partido y las aplicás sin pensar.

En programación, hay un puñado de patrones que aparecen una y otra vez. Si los aprendés de memoria, no tenés que reinventar la solución cada vez. Apenas leas el ejercicio, ya sabés qué patrón usar.

A continuación van los **5 patrones más importantes** para el parcial. Aprendelos de memoria — son los bloques con los que vas a armar el ejercicio 1.

Patrón 1: Contador

¿Para qué sirve? Contar cuántas veces pasa algo.

¿Cómo funciona? Creás una variable que arranca en 0 y le sumás 1 cada vez que pasa lo que querés contar.

```
let cantidad = 0;
for (let i = 0; i < n; i++) {
    if (cumpleAlgo) {
        cantidad++;     // suma 1 cada vez que cumple
    }
}
```

Patrón 2: Acumulador (suma)

¿Para qué sirve? Sumar todos los valores.

¿Cómo funciona? Creás una variable `suma = 0` y le vas sumando cada dato.

```
let suma = 0;
for (let i = 0; i < n; i++) {
    let dato = parseInt(prompt("Dato"));
    suma += dato;
}
```

Patrón 3: Buscar máximo

¿Para qué sirve? Encontrar el número más grande.

¿Cómo funciona? Arrancás `max` con el valor más chico posible. Por cada dato, si es mayor que el `max` actual, actualizás `max`.

```
let max = Number.MIN_SAFE_INTEGER;
for (let i = 0; i < n; i++) {
    let dato = parseInt(prompt("Dato"));
    if (dato > max) {
        max = dato;
    }
}
```

Por qué arrancar con `MIN_SAFE_INTEGER`: así garantizás que **cualquier** dato real va a ser mayor en la primera comparación. Si arrancás con 0 y todos los datos son negativos, te queda mal (todos serían menores que 0 y nunca actualizarías).

Cuándo arrancar con 0: si la letra dice que todos los datos son positivos, arrancar con 0 funciona.

Patrón 4: Buscar mínimo

Igual pero al revés:

```
let min = Number.MAX_SAFE_INTEGER;
for (let i = 0; i < n; i++) {
    let dato = parseInt(prompt("Dato"));
    if (dato < min) {
        min = dato;
    }
}
```

Patrón 5: Promedio (con cuidado de la división por 0)

¿Para qué sirve? Calcular el promedio (suma dividido cantidad).

¿Cuidado con qué? Si la cantidad es 0, dividir entre 0 da un valor raro (`Infinity` o `NaN`). Hay que evitarlo:

```
let suma = 0;
let cantidad = 0;
// ... ir contando y sumando ...

if (cantidad > 0) {
    let promedio = suma / cantidad;
    console.log("Promedio: " + promedio);
} else {
    console.log("No hay datos");
}
```

Patrón extra: bandera "hubo datos"

Cuando leés con valor centinela (por ejemplo, "0 para terminar"), capaz no entran ningún dato. Para detectar eso:

```
let hubo = false;
let dato = parseInt(prompt("Dato (0 para terminar)"));
while (dato ≠ 0) {
    hubo = true;        // marca que entró al menos uno
    // procesar dato
    dato = parseInt(prompt("Dato (0 para terminar)"));
}

if (hubo) {
    console.log("Resultado: " + resultado);
} else {
    console.log("Sin datos");
}
```

Patrón extra: asumir que cumple, buscar el primer fallo

> "¿Qué es más sencillo? ¿Decir 'esto cumple', o asumir que cumple y decir 'esto no cumple'? Para mí es más fácil buscar el error."

Cuando tenés que verificar si **todo** cumple algo, conviene **asumir que sí** y buscar el primer caso que NO cumple:

```
let ok = true;
for (let i = 0; i < texto.length && ok; i++) {
    if (noCumpleAlgo) {
        ok = false;
    }
}

if (ok) {
    alert("Válido");
} else {
    alert("No válido");
}
```

Truco del `&& ok` en la condición del for: Hace que el for **corte apenas se encuentra el primer fallo**. No sigue iterando al pedo. Optimización opcional pero buena.

PARTE 4 — Strings (cadenas de texto)

¿Qué es un string?

Un **string** es un texto. Una **cadena de caracteres**, uno detrás del otro.

Cada carácter tiene una **posición** numérica que arranca en **0** (no en 1).

```
texto = "Hola"

posición:  0  1  2  3
carácter:  H  o  l  a
```

La **última posición** siempre es `length - 1` (donde `length` es la cantidad total de caracteres).

Las dos cosas básicas: `length` y `charAt`

`length` — cuántos caracteres tiene

```
let s = "Hola";
s.length       // 4
```

¡OJO! `length` es **propiedad**, no función. **Sin paréntesis**:

```
texto.length     // bien
texto.length()   // ERROR: length no es función
```

`charAt(i)` — el carácter en la posición i

```
let s = "Hola";
s.charAt(0)    // "H"
s.charAt(1)    // "o"
s.charAt(2)    // "l"
s.charAt(3)    // "a"
s.charAt(99)   // ""  (en JS no da error, devuelve string vacío)
```

¡OJO! `charAt(i)` es **función**. **Lleva paréntesis** y un número adentro.

Recorrer un string

```
for (let i = 0; i < texto.length; i++) {
    let letra = texto.charAt(i);
    // hacer algo con letra
}
```

Por qué `i < texto.length` (estricto, no `≤`): porque la última posición válida es `length - 1`. Si ponés `≤`, te vas a una posición que no existe.

Métodos de string que hay que saber sí o sí

Cada uno de estos los explico individualmente.

`at(i)` — igual que charAt pero acepta negativos

```
"Hola".at(0)       // "H"
"Hola".at(-1)      // "a"  (último)
"Hola".at(-2)      // "l"  (penúltimo)
```

¿Para qué sirve? Para acceder a las últimas posiciones sin tener que hacer la cuenta `length - 1`.

`texto[i]` — atajo con corchetes (alternativa a `charAt`)

> Wagner mencionó esto en la última clase: como **los strings son internamente un array de caracteres**, podés acceder a las posiciones con corchetes `[ ]`, igual que en un array.

```
"Hola"[0]    // "H"   (lo mismo que "Hola".charAt(0))
"Hola"[3]    // "a"   (lo mismo que "Hola".charAt(3))
```

`texto[i]` y `texto.charAt(i)` hacen exactamente lo mismo. Usá la que te resulte más cómoda. En este documento uso `charAt(i)` por consistencia.

`toUpperCase()` y `toLowerCase()`

Pasan todo el string a mayúsculas o minúsculas.

```
"hola".toUpperCase()    // "HOLA"
"HOLA".toLowerCase()    // "hola"
"HoLa".toUpperCase()    // "HOLA"
```

¿Para qué sirve en el parcial? Para comparar strings ignorando si están en mayúsculas o minúsculas:

```
if (respuesta.toUpperCase() == "SI") { ... }
// funciona con "si", "SI", "Si", "sI"
```

`slice(inicio, fin)` — cortar un pedazo

Devuelve el pedazo del string desde `inicio` (incluido) hasta `fin` (NO incluido).

```
"Hola que tal".slice(5, 8)    // "que"
"Hola que tal".slice(5)        // "que tal"  (sin fin = hasta el final)
"Hola que tal".slice(8, 5)     // ""         (al revés, devuelve vacío)
```

Regla clave: El segundo parámetro NO se incluye. `slice(0, 4)` te da los caracteres en posición 0, 1, 2, 3 (cuatro caracteres).

`substring(inicio, fin)` — parecido a slice

```
"Hola que tal".substring(5, 8)   // "que"
"Hola que tal".substring(8, 5)   // "que"  (acomoda solo si están al
revés)
```

Diferencia con slice: `substring` se acomoda si le pasás los parámetros al revés. `slice` te devuelve vacío.

`includes(texto)` — ¿contiene este texto?

Devuelve `true` o `false`.

```
"hola que tal".includes("que")    // true
"hola que tal".includes("XYZ")    // false
"HOLA".includes("hola")            // false (es case-sensitive: distingue
mayúsculas)
```

`indexOf(texto)` — ¿en qué posición empieza?

Devuelve la posición donde aparece el texto la **primera vez**. Si no aparece, devuelve `-1`.

```
"hola que tal".indexOf("que")    // 5
"hola que tal".indexOf("XYZ")    // -1  (no lo encuentra)
"hola hola".indexOf("hola")      // 0   (primera aparición)
```

`lastIndexOf(texto)` — ¿en qué posición está la última aparición?

Como `indexOf` pero busca desde el final.

```
"hola hola".lastIndexOf("hola")   // 5  (última aparición)
```

`startsWith(texto)` — ¿empieza con este texto?

Devuelve `true` o `false`.

```
"CA12345".startsWith("CA")      // true
"AB12345".startsWith("CA")      // false
```

`endsWith(texto)` — ¿termina con este texto?

```
"hola.txt".endsWith(".txt")     // true
"hola.txt".endsWith(".pdf")     // false
```

`trim()` — saca espacios al principio y al final

```
"   hola   ".trim()             // "hola"
"   hola que tal  ".trim()      // "hola que tal" (los del medio quedan)
```

`repeat(n)` — repetir el string n veces

```
"hola".repeat(3)                // "holaholahola"
"-".repeat(20)                  // "————————————————————"
```

`concat(otro)` — concatenar (igual que `+`)

```
"hola".concat(" mundo")          // "hola mundo"
// es lo mismo que "hola" + " mundo"
```

Concepto fundamental: los strings son INMUTABLES

Inmutable = no se puede modificar. Una vez creado, un string no cambia.

> "Las funciones que yo le aplico no van a cambiar mi texto."

```
let s = "Hola";
s.toUpperCase();        // devuelve "HOLA" pero NO cambia s
console.log(s);          // "Hola"  (sigue igual)
```

Si querés cambiar la variable, hay que reasignar:

```
let s = "Hola";
s = s.toUpperCase();    // ahora SÍ se cambia
console.log(s);          // "HOLA"
```

Regla: Todos los métodos de string **devuelven un string NUEVO**. Si querés guardarlo, hay que asignarlo a una variable.

Trampa clásica: posición fuera del string

```
let texto = "Hola";
texto.charAt(10);    // ""  ← JS no se queja, pero esto puede romper tu
lógica
```

Cuando comparás dos strings de distinto largo, hay que tener cuidado de no acceder a posiciones que no existen en el más corto. Para eso se usa el truco del **circuito corto**:

```
if (i < texto.length && texto.charAt(i) == "a") { ... }
```

Si `i ≥ texto.length`, la primera condición es false, JS no evalúa la segunda, y el `if` no entra. Seguro.

PARTE 5 — Funciones

¿Qué es una función?

Una **función** es un **bloque de código con un nombre, que podés ejecutar (invocar) tantas veces como quieras**.

Analogía: una función es como una **receta nombrada**. Vos escribís "cómo hacer una pizza" una sola vez (la receta). Después, cuando querés una pizza, le decís a alguien "hacé una pizza" y se ejecuta toda la receta. Si querés tres, le decís tres veces.

¿Por qué existen?

> "Si vemos que en nuestro código aparece muchas veces el mismo bloquecito, lo llevamos a una función y lo invocamos cada vez que la necesitamos."

Sirven para **no repetir código**. Si tenés el mismo bloque de código en 5 lugares, lo metés en una función y llamás a la función desde esos 5 lugares.

Anatomía de una función

```
function calcularArea(base, altura) {
    let area = base * altura / 2;
    return area;
}
```

Cada parte:

Parte	Qué es
`function`	Palabra obligatoria que dice "voy a definir una función"
`calcularArea`	**Nombre** que vos le ponés (camelCase, mnemotécnico)
`(base, altura)`	**Parámetros formales** — variables que la función va a recibir
`{ ... }`	**Cuerpo** de la función — el código que se ejecuta
`return area`	**Sentencia de retorno** — el valor que devuelve la función

Definición vs invocación

Definir una función = escribir cómo es. Esto NO la ejecuta. **Invocar** una función = ejecutarla. Esto sí ejecuta el código.

```
// 1. DEFINICIÓN — solo describe la función, no la ejecuta
function saludar(nombre) {
    alert("Hola " + nombre);
}

// 2. INVOCACIÓN — la ejecuta
saludar("Ana");            // muestra alert "Hola Ana"
saludar("Pedro");          // muestra alert "Hola Pedro"
```

La definición se escribe **una vez**. Las invocaciones podés hacerlas **muchas**.

Parámetros formales vs parámetros reales

Esto suele confundir. Vamos despacio.

Parámetro formal

Es el **nombre** que aparece **adentro de los paréntesis cuando definís** la función.

```
function saludar(nombre) {     ← "nombre" es el parámetro formal
    alert("Hola " + nombre);
}
```

`nombre` es como una variable que **solo existe adentro** de la función. Cuando la función termina, `nombre` desaparece.

Parámetro real (o argumento)

Es el **valor** que le pasás cuando **invocás** la función.

```
saludar("Ana");                ← "Ana" es el parámetro real
```

Cuando se ejecuta, JS le da a `nombre` (el formal) el valor `"Ana"` (el real). Adentro de la función, `nombre` vale `"Ana"`.

No hace falta que tengan el mismo nombre

```
function saludar(nombre) {        // formal: nombre
    alert("Hola " + nombre);
}

let persona = "Ana";
saludar(persona);                  // real: el valor de persona ("Ana")
// adentro de la función, "nombre" toma el valor "Ana"
```

Adentro de la función, la variable se llama `nombre`. Lo que importa es **el valor que le pasaste**, no el nombre que tiene afuera.

Correspondencia por posición

Si la función tiene varios parámetros, la correspondencia es **por orden**:

```
function saludar(nombre, mensaje) {
    alert("Hola " + nombre + ", " + mensaje);
}

saludar("Ana", "buen día");
// nombre toma "Ana" (primer real → primer formal)
// mensaje toma "buen día" (segundo → segundo)
```

El `return`

```
function calcularPrecio(base, descuento) {
    let precio = base - (base * descuento / 100);
    return precio;
}

let total = calcularPrecio(1000, 10);   // total = 900
console.log(total);
```

Qué hace `return`:

1. **Termina la ejecución** de la función inmediatamente.
2. **Devuelve un valor** al que la invocó.

El `let total = calcularPrecio(1000, 10)` se traduce en:

1. Ejecuta la función con base=1000, descuento=10.
2. Adentro calcula `precio = 1000 - 100 = 900`.
3. Hace `return 900`.
4. Ese `900` es lo que se asigna a `total`.

Cualquier código después del `return` no se ejecuta.

REGLA CRÍTICA DEL CURSO: un solo `return` por función

> "En este curso vamos a tener un único `return` por función, al finalizar la función."

Si tenés más de un `return`, **te bajan puntos en el parcial**.

Mal (múltiples returns)

```
function esPar(n) {
    if (n % 2 == 0) {
        return true;
    }
    return false;
}
```

Bien (un solo return)

```
function esPar(n) {
    return n % 2 == 0;
}
```

Si necesitás más lógica, usá una variable de resultado

```
function clasificar(nota) {
    let resultado;
    if (nota ≥ 86) {
        resultado = "Exonerado";
    } else if (nota ≥ 70) {
        resultado = "Aprobado";
    } else {
        resultado = "Perdido";
    }
    return resultado;     // ← un solo return al final
}
```

Regla del profe: `return` con valores booleanos

(Esta es la versión "para funciones" de la regla que vimos en PARTE 3 sobre `if (paso == true)` vs `if (paso)`. Ahora que entendemos qué es una función y qué hace `return`, podemos ver el caso del retorno.)

El caso: Si la función va a retornar `true` o `false` según una condición, **no hace falta** escribir un `if` con asignación a una variable y después retornar la variable. Podés **retornar la condición directamente** — la condición ya devuelve `true` o `false` por sí sola.

Mal (innecesariamente largo):

```
function esArmonico(cantA, cantB) {
    let result = false;
    if (cantA == cantB) {
        result = true;
    }
    return result;
}
```

Bien (la forma corta):

```
function esArmonico(cantA, cantB) {
    return cantA == cantB;
}
```

Por qué funciona: `cantA == cantB` ya devuelve `true` cuando son iguales y `false` cuando no. No hace falta una variable intermedia ni un `if` para "convertir" eso.

Cómo identificar este patrón: Cada vez que veas que estás haciendo algo así:

```
let result = false;
if (alguna_condicion) {
    result = true;
}
return result;
```

...se puede reemplazar por:

```
return alguna_condicion;
```

Esto es algo que el profe corrige en el parcial. Hace tu código más prolijo y se nota que entendés cómo funcionan los booleanos.

Funciones que no devuelven nada

Si la función solo hace algo (como mostrar) y no necesita devolver nada, no hace falta `return`:

```
function saludar(nombre) {
    alert("Hola " + nombre);
    // sin return
}
```

Funciones que llaman a otras funciones

Una función puede invocar a otra función adentro:

```
function factorial(n) {
    let res = 1;
    for (let i = 2; i ≤ n; i++) {
        res = res * i;
    }
    return res;
}

function combinaciones(m, n) {
    return factorial(m) / (factorial(n) * factorial(m - n));
}

console.log(combinaciones(5, 2));   // 10
```

Acá `combinaciones` invoca a `factorial` tres veces. Cada invocación es independiente — JS guarda en una "pila" dónde estaba para volver después.

Scope: las variables de la función mueren cuando termina

```
function calcular(x) {
    let total = x * 2;
    return total;
}

calcular(5);
console.log(total);    // ERROR: total no existe acá afuera
```

`total` solo vive **adentro de** `calcular`. Cuando la función termina, desaparece. Lo mismo con los parámetros formales: `x` solo existe adentro.

Lo que SIEMPRE hay que hacer en el parcial

Cuando piden una función:

1. **Definir la función** con la firma exacta que pide la letra.
2. **UN solo** `return`.
3. **Después de la función, ejemplificar la invocación** con un caso concreto. Si no la ejemplificás, perdés puntos.

```
// 1. Definir
function consecutivas(texto) {
    // ...
    return tiene;
}

// 2. Ejemplificar (este renglón es OBLIGATORIO en el parcial)
let texto = prompt("Ingrese el texto");
alert(consecutivas(texto));
```

PARTE 6 — Patrones que aparecen una y otra vez en parciales

En la sección anterior (PARTE 3) introdujimos qué es un **patrón**: una forma típica de resolver un tipo de problema que se repite muchas veces. Vimos los 5 más básicos: contador, acumulador, máximo, mínimo, promedio.

En esta parte vemos los patrones **más complejos** que aparecen en los parciales: validación de entrada, lectura con valor centinela, recorrido de strings, dibujo de figuras con doble for, búsqueda del más repetido, etc.

Cada patrón tiene la misma estructura de explicación:

- **¿Para qué sirve?** — qué tipo de problema resuelve.
- **Código tipo** — la forma genérica.
- (Cuando hace falta) una corrida a mano paso a paso (qué pasa con cada variable en cada vuelta).

Patrón: Leer N datos con validación

```
let n = parseInt(prompt("¿Cuántos datos?"));
for (let i = 1; i ≤ n; i++) {
    let dato = parseInt(prompt("Dato"));
    while (dato ≤ 0) {
        alert("Inválido");
        dato = parseInt(prompt("Dato"));
    }
    // procesar dato
}
```

Lo importante: el `while` adentro del `for` valida CADA dato antes de procesarlo.

Patrón: Leer mientras el usuario quiera (con "SI"/"NO")

```
let seguir = prompt("¿Quiere ingresar otro? (SI/NO)").toUpperCase();
while (seguir == "SI") {
    // procesar
    seguir = prompt("¿Quiere ingresar otro? (SI/NO)").toUpperCase();
}
```

El `.toUpperCase()` te asegura que funciona aunque el usuario escriba "si", "Si", "SI", etc.

Patrón: Contar, sumar, máximo, mínimo (todo al mismo tiempo)

```
let cantidad = 0;
let suma = 0;
let maximo = Number.MIN_SAFE_INTEGER;
let minimo = Number.MAX_SAFE_INTEGER;

for (let i = 1; i ≤ n; i++) {
    let dato = parseInt(prompt("Dato"));
    cantidad++;
    suma += dato;
    if (dato > maximo) maximo = dato;
    if (dato < minimo) minimo = dato;
}

if (cantidad > 0) {
    console.log("Promedio: " + suma / cantidad);
    console.log("Máximo: " + maximo);
    console.log("Mínimo: " + minimo);
} else {
    console.log("Sin datos");
}
```

Patrón: Recorrer string carácter por carácter

```
function contarVocales(texto) {
    let cantidad = 0;
    for (let i = 0; i < texto.length; i++) {
        let letra = texto.charAt(i).toLowerCase();
        if (letra == "a" || letra == "e" || letra == "i" || letra == "o"
|| letra == "u") {
            cantidad++;
        }
    }
    return cantidad;
}
```

Patrón: Recorrer dos strings del mismo largo

```
function diferenciaHamming(s1, s2) {
    let cantidad = 0;
    for (let i = 0; i < s1.length; i++) {
        if (s1.charAt(i) ≠ s2.charAt(i)) {
            cantidad++;
        }
    }
    return cantidad;
}
```

Patrón: Recorrer dos strings de DISTINTO largo

Hay dos opciones.

Opción A: recorrer hasta el más corto y después agregar el resto del más largo.

```
function intercalar(s1, s2) {
    let nueva = "";
    let min = Math.min(s1.length, s2.length);
    for (let i = 0; i < min; i++) {
        nueva += s1.charAt(i) + s2.charAt(i);
    }
    if (s1.length > min) {
        nueva += s1.slice(min);
    } else {
        nueva += s2.slice(min);
    }
    return nueva;
}
```

Opción B: recorrer hasta el más largo y validar antes de acceder.

```
function intercalar(s1, s2) {
    let nueva = "";
    let max = Math.max(s1.length, s2.length);
    for (let i = 0; i < max; i++) {
        if (i < s1.length) nueva += s1.charAt(i);
        if (i < s2.length) nueva += s2.charAt(i);
    }
    return nueva;
}
```

Patrón: Doble for para imprimir patrones (Ejercicio 3 del parcial)

Estrategia general

1. **Identificá la cantidad de filas** → ese es el `for` exterior.
2. **Identificá qué hay en cada fila** → ese es el `for` interior (o varios for interiores seguidos).
3. **Construí cada línea como un string vacío e ir concatenando**.
4. Al final de la fila exterior, hacés `console.log(linea)`.

```
for (let i = 1; i ≤ cantidadFilas; i++) {
    let linea = "";
    for (let j = ... ; ... ; ... ) {
        linea += loQueVa;
    }
    console.log(linea);
}
```

Ejemplo: triángulo de asteriscos

Salida esperada (n=5):

```
*
**
***
****
*****
```

Código:

```
for (let i = 1; i ≤ n; i++) {
    let linea = "";
    for (let j = 1; j ≤ i; j++) {
        linea += "*";
    }
    console.log(linea);
}
```

Corrida a mano con n=3:

i (fila)	j va de 1 a i	linea final
1	j=1	"*"
2	j=1, j=2	"**"
3	j=1, j=2, j=3	"***"

Ejemplo: cuadrado dividido en dos triángulos

Salida (n=5, carácter "?"):

```
?????
$????
<div class="katex-display"><span class="katex-display"><span class="katex">
<span class="katex-mathml"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="block"><semantics><mrow><mo stretchy="false">?</mo><mo
stretchy="false">?</mo><mo stretchy="false">?</mo></mrow><annotation
encoding="application/x-tex">???</annotation></semantics></math></span><span
class="katex-html" aria-hidden="true"><span class="base"><span class="strut"
style="height:0.6944em;"></span><span class="mclose">???</span></span></span>
</span></span></div>$??
<div class="katex-display"><span class="katex-display"><span class="katex">
<span class="katex-mathml"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="block"><semantics><mrow></mrow><annotation encoding="application/x-
tex"></annotation></semantics></math></span><span class="katex-html" aria-
hidden="true"></span></span></span></div>?
```

Código:

```javascript
for (let i = 1; i ≤ n; i++) {
    let linea = "";
    for (let j = 1; j ≤ i - 1; j++) {
        linea += "$";
    }
    for (let j = i; j ≤ n; j++) {
        linea += "?";
    }
    console.log(linea);
}
```

En cada fila hay **dos for internos**: uno para los `$`, otro para los `?`. En la fila `i` hay `i-1` dólares y `n-i+1` interrogaciones.

Ejemplo: línea espejada

Salida (n=6):

```
1 2 3 3 2 1
1 2 2 1
1 1
```

Código:

```
let topeLinea = n / 2;
for (let i = 1; i ≤ n / 2; i++) {
    let linea = "";
    for (let j = 1; j ≤ topeLinea; j++) {
        linea += j + " ";
    }
    for (let j = topeLinea; j ≥ 1; j--) {
        linea += j + " ";
    }
    topeLinea--;
    console.log(linea);
}
```

Acá el segundo for interno **va al revés** (de `topeLinea` a 1), por eso usa `j--`.

Patrón: Verificar si todo cumple

Truco potente:

```
function todoMayuscula(texto) {
    return texto == texto.toUpperCase();
}
```

Por qué funciona: Si el texto ya está todo en mayúsculas, pasarlo a mayúsculas no lo cambia. Entonces son iguales.

Patrón: Buscar el más repetido

```
function letraMasRepetida(texto) {
    texto = texto.toUpperCase();
    let max = 0;
    let letraMax = "";
    for (let i = 0; i < texto.length; i++) {
        let letra = texto.charAt(i);
        let cant = 0;
        for (let j = 0; j < texto.length; j++) {
            if (texto.charAt(j) == letra) {
                cant++;
            }
        }
        if (cant > max) {
            max = cant;
            letraMax = letra;
        }
    }
    return letraMax;
}
```

Cómo funciona: Por cada letra del texto (for exterior), cuenta cuántas veces aparece esa letra en todo el texto (for interior). Si esa cantidad supera el máximo actual, actualiza.

Patrón: Acumular hasta superar un límite (estilo "camión de paquetes")

¿Para qué sirve?

Cuando recibís una secuencia de elementos y los vas **agregando a un grupo** hasta que el grupo se llena, ahí cerrás el grupo y empezás uno nuevo. **El orden de los elementos importa** — no podés saltarte uno y dejarlo para después.

Ejemplos del estilo:

- Repartir paquetes en viajes, sin superar la carga máxima del camión.
- Agrupar palabras en líneas que no superen cierto largo.

- Repartir gastos en cuotas que no superen un tope mensual.

Código tipo

```
let cantGrupos = 1;        // arrancamos con un grupo abierto
let acumulado = 0;         // lo que llevo en el grupo actual

for (let i = 0; i < n; i++) {
    let elemento = parseInt(prompt("Elemento"));
    if (acumulado + elemento ≤ limite) {
        // entra en el grupo actual
        acumulado += elemento;
    } else {
        // no entra: cerramos este grupo y arrancamos uno nuevo
        cantGrupos++;
        acumulado = elemento;   // el nuevo grupo arranca con este
elemento
    }
}
```

Ejemplo concreto: camión de paquetes

Letra: Un camión tiene una carga máxima. Recibís paquetes de a uno (con su peso) y los vas cargando en orden. Cuando uno no entra, cerrás el viaje y arrancás uno nuevo. Calcular cuántos viajes en total. Asumir que ningún paquete por sí solo supera el máximo.

```
function viajes(cantPaquetes, cargaMax) {
    let cantViajes = 1;
    let cargaViaje = 0;
    for (let i = 0; i < cantPaquetes; i++) {
        let paquete = parseInt(prompt("Peso del paquete"));
        if (cargaViaje + paquete ≤ cargaMax) {
            cargaViaje += paquete;
        } else {
            cantViajes++;
            cargaViaje = paquete;
        }
    }
    return cantViajes;
}

console.log(viajes(5, 100));
```

Corrida a mano

Con `cargaMax = 10` y paquetes `5, 4, 3, 2, 1, 8`:

Paquete	cargaViaje + paquete	¿≤ 10?	Acción	cargaViaje	cantViajes
inicio	—	—	—	0	1
5	0+5 = 5	sí	suma	5	1
4	5+4 = 9	sí	suma	9	1
3	9+3 = 12	**no**	nuevo	3	2
2	3+2 = 5	sí	suma	5	2
1	5+1 = 6	sí	suma	6	2
8	6+8 = 14	**no**	nuevo	8	3

Resultado: **3 viajes** (5+4 / 3+2+1 / 8).

Lo crítico de este patrón

1. `cantViajes` **arranca en 1**, no en 0. Si hay al menos un paquete, hay al menos un viaje.
2. **Cuando no entra**, no se "salta" el paquete: se mete en el nuevo viaje. Por eso `cargaViaje = paquete` (no `cargaViaje = 0`).
3. **El orden importa**: no podés "guardar" un paquete que no entró para meterlo después en otro viaje donde sí entraría. Tenés que cerrarlo donde está.

PARTE 7 — Errores comunes y consejos del profesor

Errores que el profe vio una y otra vez

Error 1: olvidarse del `parseInt`

```
let n = prompt("Número");
n + 5     // "55" si pusieron 5  ← MAL: concatena
parseInt(prompt("Número")) + 5   // 10  ← bien
```

Error 2: usar `=` en vez de `==`

```
if (x = 5) { ... }     // ESTO ASIGNA 5 a x. La condición casi siempre es
true.
if (x == 5) { ... }    // bien, compara
```

Error 3: acceder fuera del string

```
for (let i = 0; i ≤ texto.length; i++) {   // ≤ en vez de <
    texto.charAt(i)    // en la última iteración, i == length, posición
que no existe
}
```

Error 4: dividir entre 0

```
let promedio = suma / cantidad;    // si cantidad es 0, da NaN
// SOLUCIÓN: validar antes
if (cantidad > 0) {
    promedio = suma / cantidad;
}
```

Error 5: olvidarse de modificar la variable del while

```
let dato = parseInt(prompt("Dato"));
while (dato ≠ 0) {
    suma += dato;
    // ¡¡falta volver a pedir dato!! → loop infinito
}
```

Correcto:

```
let dato = parseInt(prompt("Dato"));
while (dato ≠ 0) {
    suma += dato;
    dato = parseInt(prompt("Dato"));   // ← clave
}
```

Error 6: comparar mayúscula con minúscula

```
"Hola" == "hola"    // false  ← cuidado
```

Solución:

```
"Hola".toUpperCase() == "hola".toUpperCase()    // true
```

Error 7: confundir `length` con `length()`

```
texto.length        // bien (sin paréntesis, es propiedad)
texto.length()      // ERROR
texto.charAt(0)     // bien (con paréntesis, es método)
texto.charAt        // ERROR (le falta paréntesis)
```

Error 8: límite del for mal puesto

```
for (let i = 0; i ≤ n; i++)    // ejecuta n+1 veces, NO n
for (let i = 0; i < n; i++)    // ejecuta n veces (correcto)
for (let i = 1; i ≤ n; i++)    // también ejecuta n veces (correcto)
```

Error 9: no validar entrada

Si la letra dice "ingresar hasta que sea positivo, sino reingresar", hay que validar con un `while`. No se puede asumir.

Error 10: usar `var` en vez de `let`

`var` está prohibido. Solo `let` (y `const` para constantes).

Consejos directos del profesor

> "Solución simple. No se compliquen."

> "Lean bien la letra. Si no entienden algo, pregunten."

> "Hagan corridas a mano en el borrador con ejemplos chicos."

> "Empiecen por el ejercicio que les sale primero. No tienen que hacer en orden."

> "Si encuentran un patrón, piensen si vale la pena cortar el for con `&& ok`. A veces sí, a veces no."

> "El `for` y el `while` se pueden convertir uno en otro. Cualquiera funciona."

> "Cuando definan una variable solo para retornarla, mejor retornen directo la expresión."

> "Las funciones tienen UN solo `return`. Si tienen dos, pierden puntos."

> "Pongan punto y coma al final de cada sentencia, aunque en JS no sea obligatorio."

> "Indenten bien el código. Un código mal indentado les puede costar puntos."

Consejos finales (clase 16, día antes del parcial)

En la última clase antes del parcial, Wagner repasó las **buenas prácticas** y dio consejos prácticos:

Buenas prácticas (te restan puntos si no las cumplís)

- **Nombres de variables mnemotécnicos**, en minúscula, con camelCase si son varias palabras (`puntajeMaximo`, no `pm` ni `puntaje_maximo`).
- **Inicializar las variables al principio**, antes de usarlas.
- **Usar `let`**, nunca `var`.
- **Para comparar usar `==` o `===`, NUNCA `=`** (el `=` es asignación). Confusión clásica: `if (edad = 18)` ASIGNA 18 a `edad` y siempre da true. Lo correcto es `if (edad == 18)`.
- **Con booleanos NO comparar contra true/false**: usar `if (terminado)` en vez de `if (terminado == true)`. Y `if (!terminado)` en vez de `if (terminado == false)`.
- **Punto y coma al final de cada sentencia.**
- **Un único `return` al final de cada función.**

Pensar como robot, no como humano

> "Ustedes piénsenlo como que son un robotito y van a tener que ir haciendo ciertas acciones. No que lo hagan así rápido como con el cerebro humano. Que vayan haciendo procesos."

Aplicación práctica: Cuando lees el ejemplo de la letra del parcial, **leelo como si vos fueras la computadora ejecutando paso a paso**. Esa lectura te muestra qué hay que hacer en cada vuelta del bucle. Después llevás eso a código.

Ejemplo: si la letra dice "se reciben los paquetes 5, 4, 3, 2, 1, 8 y la carga máxima es 10", leé:

- "Llega el 5. La carga del viaje es 0. ¿Entra? Sí. Carga = 5."
- "Llega el 4. ¿9 ≤ 10? Sí. Carga = 9."
- "Llega el 3. ¿12 ≤ 10? No. Cierro este viaje, abro uno nuevo. Carga = 3. Viajes = 2."
- ... y así.

Cuando termines de leer así el ejemplo, ya sabés EXACTAMENTE qué tiene que hacer tu código.

No siempre necesitás un `for`

> "A veces se maten, empiezan con la idea de recorrer y la idea de recorrer les vuela la cabeza porque tienen que hacer un montón de casuísticas y les complica, y había una solución mucho más sencilla sin necesidad de recorrer."

Antes de empezar a recorrer con un for, preguntate:

- ¿Hay alguna función ya hecha que resuelve esto? (`parseInt`, `slice`, `includes`, `toUpperCase`, `Math.max`, etc.)
- ¿Puedo combinar dos cosas en una sola? (ejemplo: concatenar dos strings antes de recorrer, en vez de recorrer cada uno por separado)

Durante el parcial, podés preguntar dudas de letra

> "Durante el parcial pueden hacer cualquier duda de letra."

Si algo no se entiende de la letra, **preguntale al docente**. No te quedes con la duda. Lo único que NO podés preguntar es cómo resolverlo.

¿Qué es una "corrida a mano"?

Una **corrida a mano** es **simular en papel lo que el código hace, paso a paso**, anotando cómo cambian las variables.

Sirve para verificar que tu código funciona antes de dar por terminado el ejercicio.

Ejemplo de corrida a mano

Código:

```
let suma = 0;
for (let i = 1; i ≤ 3; i++) {
    let dato = i * 10;
    suma += dato;
}
```

Corrida a mano:

Paso	i	dato	suma
Inicio	—	—	0
Vuelta 1 (i=1)	1	10	0+10 = 10
Vuelta 2 (i=2)	2	20	10+20 = 30
Vuelta 3 (i=3)	3	30	30+30 = 60
Fin (i=4, no entra)	4	—	60

Después de la corrida, sé que `suma` queda en 60.

Hacé esto con cualquier ejercicio del parcial. Si el resultado no coincide con lo que esperabas, hay un bug.

PARTE 8 — Checklist pre-parcial

Antes de entregar cada ejercicio, revisar:

Generales

- ☐ **¿Lo que escribí responde lo que pide la letra?** Releer la letra.
- ☐ **¿Probé con los ejemplos que da la letra?** Hacer corrida a mano.
- ☐ **¿Probé con casos borde?** (vacío, un solo elemento, todo igual, todos cero, todos negativos)
- ☐ **¿Cada sentencia termina en `;`?**
- ☐ **¿Las llaves `{}` están bien apareadas?**
- ☐ **¿La indentación es consistente?**
- ☐ **¿Los nombres de variables son claros?** (no `a`, `b`, `x`)
- ☐ **¿Está todo en `let`?** (no `var`)

Para programas enteros (Ejercicio 1)

- ☐ **¿Parseo todos los `prompt`?** (parseInt si es número entero, parseFloat si tiene decimales)
- ☐ **¿Validé las entradas según pide la letra?** (mientras sea negativo, vuelvo a pedir)
- ☐ **¿Manejé el caso "sin datos"?** (si no se ingresó nada, evitar dividir por 0)
- ☐ **¿Inicialicé bien los acumuladores?** (`suma = 0`, `cantidad = 0`, `max = MIN_SAFE_INTEGER`, `min = MAX_SAFE_INTEGER`)
- ☐ **¿El último mensaje sale por `console.log` o `alert`?** (lo que pida la letra)

Para funciones (Ejercicio 2)

- ☐ **¿Tiene un solo `return`?**
- ☐ **¿La firma coincide con lo que pide la letra?** (nombre, parámetros)
- ☐ **¿Devuelve el tipo correcto?** (número, string, booleano)
- ☐ **¿Ejemplifiqué la invocación?** (en el código, abajo de la función)
- ☐ **¿Manejé mayúsculas/minúsculas si corresponde?**

Para patrones (Ejercicio 3)

- ☐ **¿Cuento bien las filas?** (¿de 1 a n? ¿de 0 a n-1?)
- ☐ **¿Cada fila se construye como string y se imprime al final con `console.log(linea)`?**
- ☐ **¿Los espacios coinciden con el ejemplo?** (mirar bien si hay espacios al principio)
- ☐ **¿Probé con n=1 y n=2 a mano?**

PARTE 9 — Ejercicios resueltos del estilo del parcial

En esta parte hay **10 ejercicios resueltos**:

- **3 ejercicios "DESTACADOS"** (Ej1★, Ej2★, Ej3★): los 3 que hizo Wagner en la última clase antes del parcial (clase 16, 27/04). Son los más relevantes porque muestran exactamente el formato del parcial de mañana.
- **7 ejercicios** sacados de parciales reales de años anteriores (2024–2025) para más práctica.

Cómo usar esta sección

1. **Leé la letra** del ejercicio.
2. **Tapá la solución** y trata de hacerlo en papel.
3. **Después comparalo** con la solución dada.
4. **Leé la sección "Lo que hay que mirar"** abajo de cada solución — explica qué decisiones se tomaron y por qué.

Codificación de los ejercicios

- "★" indica los 3 ejercicios DESTACADOS de la clase 16 (los más relevantes para mañana).

- "**Ejercicio 1A, 1B, 1C**" son del **estilo del Ejercicio 1 del parcial** (programa entero, 7 puntos, con `prompt` / `alert` / `console.log`).
- "**Ejercicio 2A, 2B**" son del **estilo del Ejercicio 2** (función, 5 puntos).
- "**Ejercicio 3A, 3B**" son del **estilo del Ejercicio 3** (programa o función más complejo, 3 puntos).

★ Ejercicio 1★ — Estadísticas de torneo de tenis (clase 16)

Letra: Programa en JS para estadísticas de un torneo de tenis con más de 50 jugadores. Se juegan dos rondas; el puntaje total de cada jugador es la suma de sus dos rondas. De cada participante se ingresa: nombre, puntaje en la primera ronda, puntaje en la segunda ronda. **Los datos no necesitan validarse.** El fin de ingreso está dado por el nombre `"fin"` y ambos puntajes en `0`.

Al final mostrar:

- Cuántos jugadores mejoraron su desempeño de la primera a la segunda ronda.
- Cuál fue el puntaje total máximo del torneo.
- Nombre de **un** jugador que obtuvo ese puntaje máximo (cualquiera si hay varios).
- Cantidad de jugadores que obtuvieron ese puntaje máximo.

Solución:

```
let mejoro = 0;
let puntajeMaximo = 0;
let nombreMaximo = "";
let cantMaximo = 0;

let nombre = prompt("Nombre");
let p1 = parseInt(prompt("Primer puntaje"));
let p2 = parseInt(prompt("Segundo puntaje"));

while (nombre ≠ "fin" && p1 ≠ 0 && p2 ≠ 0) {
    if (p2 > p1) {
        mejoro++;
    }
    if (p1 + p2 > puntajeMaximo) {
        puntajeMaximo = p1 + p2;
        nombreMaximo = nombre;
        cantMaximo = 1;
    } else if (p1 + p2 == puntajeMaximo) {
        cantMaximo++;
    }
    nombre = prompt("Nombre");
    p1 = parseInt(prompt("Primer puntaje"));
    p2 = parseInt(prompt("Segundo puntaje"));
}

alert("Mejoraron: " + mejoro);
alert("Puntaje máximo: " + puntajeMaximo);
alert("Nombre: " + nombreMaximo);
alert("Cantidad con máximo: " + cantMaximo);
```

Lo que hay que mirar (es CRÍTICO entenderlo)

1. **Variables iniciales**: 4 acumuladores (`mejoro`, `puntajeMaximo`, `nombreMaximo`, `cantMaximo`). Como los puntajes no son negativos, `puntajeMaximo` arranca en `0` (no hace falta `MIN_SAFE_INTEGER`).
2. **Pedir datos antes del while** y **volver a pedir al final del while**: clásico patrón de centinela. La condición de salida es "nombre es fin Y los dos puntajes son 0". Con que **una sola** falle, el while termina.
3. **El IF de "encontré un nuevo máximo"**: cuando aparece un puntaje total mayor al máximo actual, **pisás todo**:

- `puntajeMaximo` = nuevo máximo.
- `nombreMaximo` = nombre del jugador actual.
- `cantMaximo = 1` ← ¡UNO, no le sumás 1! Porque los jugadores anteriores tenían un puntaje menor que ya no es el máximo. **Esto es lo más confuso del ejercicio**, releé la corrida en la clase 16 para entenderlo.

4. **El ELSE IF de "encontré OTRO con el mismo máximo"**: solo cuando el puntaje total es **igual** al máximo actual, sumás 1 a `cantMaximo`. Tiene que ser ELSE IF (no IF separado), porque si no, cuando entrás al primer IF (nuevo máximo) también entraría al segundo (porque acabás de igualar al máximo).
5. `p1 + p2` se calcula varias veces — podrías guardarlo en una variable `let total = p1 + p2;` para no repetir.

★ Ejercicio 2★ — Verificar dieta diaria (clase 16)

Letra: En cierta dieta solo se permite comer alimentos representados por letras mayúsculas. Programa en JS que solicita 3 strings: la dieta, el almuerzo, y la cena. Indica si se cumplió la dieta o no. Se asume todo en mayúsculas, sin validación.

Ejemplo:

- Dieta: `"BNACDE"`, Almuerzo: `"AEF"`, Cena: `"BBCABJ"` → **No cumple** (la F del almuerzo y la J de la cena no están en la dieta).
- Dieta: `"BTEAG"`, Almuerzo: `"GGGGGT"`, Cena: `"BA"` → **Cumple**.

Solución:

```
let dieta = prompt("Dieta");
let almuerzo = prompt("Almuerzo");
let cena = prompt("Cena");

let comio = almuerzo + cena;        // truco: concatenar para recorrer una
sola vez
let ok = true;

for (let i = 0; i < comio.length && ok; i++) {
    if (!dieta.includes(comio.charAt(i))) {
        ok = false;
    }
}

if (ok) {
    alert("Cumplió");
} else {
    alert("No cumplió");
}
```

Lo que hay que mirar

1. **Truco clave: concatenar** `almuerzo + cena` en una sola variable `comio`. Como NO importa en qué momento del día se comió cada cosa (solo importa que TODO esté en la dieta), recorrer una sola string es más simple que recorrer dos por separado.
2. **Asumir que cumple, buscar el contraejemplo**: arrancás con `ok = true`. Recorrés cada letra de `comio`. Si encontrás una que no está en la dieta, `ok = false`.
3. `!dieta.includes(comio.charAt(i))`: el `!` invierte. La condición se cumple cuando la letra **NO** está en la dieta. Ahí entra al if y marca `ok = false`.
4. `&& ok` **en la condición del for**: corta el for apenas encuentra el primer fallo. Optimización buena.
5. **Cuidado con strings de distinto largo**: si recorriéramos `almuerzo` y `cena` por separado con un solo `i`, podríamos acceder a posiciones que no existen en uno de los dos. Concatenar evita ese problema.

★ Ejercicio 3★ — Camión de paquetes (clase 16)

Letra: Función `viajes(cantPaquetes, cargaMax)` que recibe la cantidad de paquetes a entregar y la carga máxima del camión. Dentro de la función pide el peso de cada paquete. Los paquetes deben entregarse **en el orden recibido**. Ningún paquete supera la carga máxima por sí solo. Retorna la cantidad mínima de viajes necesarios.

Ejemplo: `cantPaquetes = 6`, `cargaMax = 10`, paquetes `5, 4, 3, 2, 1, 8` → 3 viajes (5+4 / 3+2+1 / 8).

Solución:

```
function viajes(cantPaquetes, cargaMax) {
    let cantViajes = 1;
    let cargaViaje = 0;
    for (let i = 0; i < cantPaquetes; i++) {
        let paquete = parseInt(prompt("Peso del paquete"));
        if (cargaViaje + paquete ≤ cargaMax) {
            cargaViaje += paquete;
        } else {
            cantViajes++;
            cargaViaje = paquete;
        }
    }
    return cantViajes;
}

console.log(viajes(5, 100));    // invocación pedida en la letra
```

Lo que hay que mirar

1. **Parámetros vienen de afuera**: la letra dice "recibe el peso máximo y la cantidad de paquetes como parámetro". O sea: NO los pidas con prompt. Vienen ya como `cantPaquetes` y `cargaMax` desde la invocación.
2. **`cantViajes` arranca en 1, no en 0**: la letra dice que la cantidad de paquetes es ≥ 1, así que va a haber al menos un viaje sí o sí.
3. **El IF "el paquete entra"**: si la suma de lo que ya cargué más este nuevo paquete no supera el máximo, lo agrego al viaje actual.

4. **El ELSE "el paquete no entra, abro nuevo viaje"**: incremento `cantViajes` Y reinicio `cargaViaje = paquete` (NO en 0). El nuevo viaje arranca con este paquete, no vacío.
5. **Invocación con `console.log`**: la letra decía "agregar la invocación por consola". Eso significa usar `console.log`, no `alert`. **Leé bien la letra** — siempre te dice cómo mostrar.
6. **Refleja el patrón "acumular hasta superar un límite"** que vimos en PARTE 6.

Ejercicio 1A — Estilo Ej1: Programa entero

Letra: Una máquina produce tornillos. Se ingresa la medida oficial (positiva) y luego 500 medidas. Si una medida es ≤ 0, reingresar. Mostrar cuántos miden exactamente la medida oficial y el rango (max - min).

Solución:

```javascript
let cantidad = 0;
let min = Number.MAX_SAFE_INTEGER;
let max = 0;
let medidaOficial = parseInt(prompt("Ingrese la medida oficial"));

for (let i = 1; i ≤ 500; i++) {
    let medida = parseInt(prompt("Medida del tornillo"));
    while (medida ≤ 0) {
        alert("Inválida, reingrese");
        medida = parseInt(prompt("Medida del tornillo"));
    }
    if (medida == medidaOficial) {
        cantidad++;
    }
    if (medida < min) {
        min = medida;
    }
    if (medida > max) {
        max = medida;
    }
}

console.log("Cantidad oficiales: " + cantidad);
console.log("Rango: " + (max - min));
```

Lo que hay que mirar

1. `min` **arranca en** `MAX_SAFE_INTEGER`, así cualquier medida real va a ser menor en la primera comparación.
2. `max` **arranca en 0**, está bien porque las medidas son positivas. Si pudieran ser negativas, habría que arrancar en `MIN_SAFE_INTEGER`.
3. **Validación con** `while` — clásica.
4. **Tres** `if` **separados** dentro del for: contar oficiales, actualizar min, actualizar max. Son independientes, no se pisan.

Ejercicio 2A — Estilo Ej2: Función

Letra: Implementar función `consecutivas(texto)` que recibe un string y retorna `true` si tiene 3 o más posiciones consecutivas con el mismo carácter, o `false` si no.

Ejemplos:

- `"hoooola"` → true (las tres "o" del medio)
- `"494944442333"` → true (los cuatro "4" del medio)
- `"212Aaa12312"` → false (Aaa son distintas porque "A" mayúscula ≠ "a" minúscula)

Solución:

```
function consecutivas(texto) {
    let tiene = false;
    for (let i = 0; i < texto.length - 2 && !tiene; i++) {
        if (texto.charAt(i) == texto.charAt(i + 1) &&
            texto.charAt(i) == texto.charAt(i + 2)) {
            tiene = true;
        }
    }
    return tiene;
}

// Ejemplificar (OBLIGATORIO):
let texto = prompt("Ingrese texto");
alert(consecutivas(texto));
```

Lo que hay que mirar

1. `i < texto.length - 2`: como compara `i`, `i+1`, `i+2`, el último `i` válido es `length - 3`, o sea menor que `length - 2`.
2. `&& !tiene`: corta el for apenas encuentra el primer caso. Optimización opcional pero buena.
3. **Un solo** `return` al final.
4. **Ejemplifico la invocación** abajo de la función.

Ejercicio 3A — Estilo Ej3: Patrón con doble for

Letra: Leer p y q. Mostrar p filas de números consecutivos desde 1, alternando q y q-1 números por fila. Las filas con q-1 números llevan 2 espacios al inicio.

Ejemplo p=3, q=5:

```
1 2 3 4 5
  6 7 8 9
10 11 12 13 14
```

Solución:

```
let p = parseInt(prompt("p"));
let q = parseInt(prompt("q"));
let numero = 1;

for (let i = 1; i ≤ p; i++) {
    let lin;
    if (i % 2 == 0) {
        // fila par: q-1 números, con 2 espacios al inicio
        lin = "  ";
        for (let j = 1; j < q; j++) {
            lin += numero + " ";
            numero++;
        }
    } else {
        // fila impar: q números
        lin = "";
        for (let j = 1; j ≤ q; j++) {
            lin += numero + " ";
            numero++;
        }
    }
    console.log(lin);
}
```

Lo que hay que mirar

1. `numero` **se mantiene afuera del for principal** porque tiene que ir aumentando entre filas.

2. `i % 2` decide qué tipo de fila es.
3. `lin = " "` con dos espacios para las filas con menos números.
4. `j < q` vs `j ≤ q` según cuántos números van en cada tipo de fila.
5. Cada `console.log(lin)` imprime una fila entera.

Ejercicio 1B — Estilo Ej1: Plan de natación

Letra: Ingresar total de metros y total de sesiones objetivo. Luego, mientras el usuario quiera, ingresar metros de cada sesión. Al final, decir:

- "Felicitaciones" (cumplió ambas metas)
- "Buen trabajo en distancia!" (cumplió metros pero no sesiones)
- "Buen trabajo en sesiones!" (cumplió sesiones pero no metros)
- "Continúa practicando!" (no cumplió ninguno)

Y mostrar el promedio de metros por sesión.

Solución:

```
let totalMetros = parseInt(prompt("Total metros objetivo"));
let totalSesiones = parseInt(prompt("Total sesiones objetivo"));
let metros = 0;
let sesiones = 0;
let seguir = "SI";

while (seguir == "SI") {
    let m = parseInt(prompt("Metros de esta sesión"));
    metros += m;
    sesiones++;
    seguir = prompt("¿Quiere ingresar otra? (SI/NO)").toUpperCase();
}

if (metros ≥ totalMetros && sesiones ≥ totalSesiones) {
    console.log("Felicitaciones!");
} else if (metros ≥ totalMetros) {
    console.log("Buen trabajo en distancia!");
} else if (sesiones ≥ totalSesiones) {
    console.log("Buen trabajo en sesiones!");
} else {
    console.log("Continúa practicando!");
}

console.log("Promedio: " + metros / sesiones);
```

Lo que hay que mirar

1. `while` **con valor centinela "SI"** para entrada repetida.
2. `.toUpperCase()` para no preocuparse de mayúsculas.
3. **Cadena** `if / else if / else` para los 4 casos en orden de prioridad.
4. **Acumular** `metros` **y** `sesiones` **mientras se pide**.

Ejercicio 2B — Estilo Ej2: Función `coinciden`

Letra: Función `coinciden(palabra)` que retorna true si "one" aparece la misma cantidad de veces que "HI" (ignorando mayúsculas).

Solución:

```
function coinciden(palabra) {
    palabra = palabra.toUpperCase();
    let cantONE = 0;
    let cantHI = 0;
    for (let i = 0; i < palabra.length - 2; i++) {
        if (palabra.substring(i, i + 3) == "ONE") {
            cantONE++;
        }
        if (palabra.substring(i, i + 2) == "HI") {
            cantHI++;
        }
    }
    // Puede haber un HI en las últimas 2 posiciones que el for no chequea
    if (palabra.substring(palabra.length - 2) == "HI") {
        cantHI++;
    }
    return cantONE == cantHI;
}

alert(coinciden("hiOneComputerTWOdays"));
```

Lo que hay que mirar

1. **Pasar todo a mayúsculas** al inicio simplifica la comparación.
2. `substring(i, i+3)` para extraer 3 caracteres consecutivos.
3. **Cuidado con el último HI** que puede quedar fuera del bucle principal.
4. **Retorno directo de la comparación** (no `if (cantONE == cantHI) return true; else return false;`).

Ejercicio 3B — Estilo Ej3: Cuadrado de dos triángulos

Letra: Leer n y un carácter. Mostrar un cuadrado de lado n donde el triángulo superior derecho usa ese carácter y el inferior izquierdo usa "$".

Ejemplo n=5, carácter= `?` :

```
?????
$????
<div class="katex-display"><span class="katex-display"><span class="katex">
<span class="katex-mathml"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="block"><semantics><mrow><mo stretchy="false">?</mo><mo
stretchy="false">?</mo><mo stretchy="false">?</mo></mrow><annotation
encoding="application/x-tex">???</annotation></semantics></math></span><span
class="katex-html" aria-hidden="true"><span class="base"><span class="strut"
style="height:0.6944em;"></span><span class="mclose">???</span></span></span>
</span></span></div>$??
<div class="katex-display"><span class="katex-display"><span class="katex">
<span class="katex-mathml"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="block"><semantics><mrow></mrow><annotation encoding="application/x-
tex"></annotation></semantics></math></span><span class="katex-html" aria-
hidden="true"></span></span></span></div>?
```

Solución:

```javascript
let n = parseInt(prompt("n"));
let carac = prompt("carácter");

for (let i = 1; i ≤ n; i++) {
    let linea = "";
    for (let j = 1; j ≤ i - 1; j++) {
        linea += "$";
    }
    for (let j = i; j ≤ n; j++) {
        linea += carac;
    }
    console.log(linea);
}
```

Lo que hay que mirar

1. **Dos for internos** dentro del for de filas: uno para los `$`, otro para el carácter.
2. **En la fila i**: hay `i-1` dólares y `n-i+1` caracteres.
3. **Fila 1**: 0 `$`, n caracteres.
4. **Fila n**: n-1 `$`, 1 carácter.

Ejercicio 1C — Estilo Ej1: Asistencia (string como input)

Letra: Recibir string de asistencia (1=vino, 0=no). Mostrar:

- "excelente asistencia" (sin ningún 0)
- "muy buena asistencia" (vino primer y último día, y solo una falta)
- "asistencia a mejorar" (cualquier otro caso)

Solución:

```
let lista = prompt("Asistencias");

if (!lista.includes("0")) {
    alert("excelente asistencia");
} else {
    let cant0 = 0;
    for (let i = 0; i < lista.length; i++) {
        if (lista.charAt(i) == "0") {
            cant0++;
        }
    }
    if (cant0 == 1 && lista.charAt(0) == "1" && lista.charAt(lista.length
- 1) == "1") {
        alert("muy buena asistencia");
    } else {
        alert("asistencia a mejorar");
    }
}
```

Lo que hay que mirar

1. **Verificar primero el caso "no hay ningún 0"** con `includes`. Si entra, listo.
2. **Si hay al menos un 0, contarlos** con un for.
3. **Para "muy buena asistencia"**: 3 condiciones simultáneas.

PARTE 10 — Preguntas de práctica

Conceptuales

1. ¿Qué diferencia hay entre `==` y `===`?

`==` compara solo el valor (con conversión automática de tipos). `===` compara valor Y tipo. `5 == "5"` es true, `5 === "5"` es false.

2. ¿Qué hace `Number.MAX_SAFE_INTEGER`?

Es el entero positivo más grande que JS puede representar con seguridad. Se usa para inicializar variables que van a buscar el mínimo.

3. ¿Por qué `let` y no `var`?

`let` tiene scope de bloque (solo vive donde se declaró). `var` tiene scope de función o global, lo que provoca bugs.

4. ¿Qué pasa si un `while` nunca termina su condición?

Loop infinito: el programa se cuelga. Hay que asegurar que dentro del bloque haya algo que eventualmente vuelva la condición falsa.

5. ¿Qué garantiza un `MIENTRAS` cuando termina?

Que la condición es falsa. Es la única forma de que termine.

6. ¿Qué diferencia hay entre `MIENTRAS` y `REPETIR ... MIENTRAS`?

`MIENTRAS` puede ejecutarse 0 veces si la condición es falsa desde el inicio. `REPETIR ... MIENTRAS` siempre ejecuta el bloque al menos una vez.

7. ¿Qué devuelve `prompt`?

Siempre un string. Si querés trabajar con números, hay que `parseInt` o `parseFloat`.

8. ¿Cómo se calcula el resto de la división?

Con el operador `%` (módulo). `15 % 4` es 3.

9. ¿Qué imprime `"3" + 4 + 5`?

`"345"` — porque `"3" + 4` da `"34"` (concatena), y después `"34" + 5` da `"345"`.

10. ¿Qué imprime `3 + 4 + "5"`?

`"75"` — porque `3 + 4` da `7` (suma), y después `7 + "5"` da `"75"` (concatena).

11. ¿Cuántos `return` debe tener una función en este curso?

Uno solo, al final.

12. ¿La propiedad `length` de un string lleva paréntesis?

No. `length` es propiedad, no método. `texto.length`.

13. ¿Qué valor está en la posición 0 de un string?

El primer carácter. Los strings se indexan desde 0.

14. ¿Qué devuelve `slice(3, 7)`?

Los caracteres desde la posición 3 (inclusive) hasta la 7 (exclusive). En total 4 caracteres.

15. ¿Qué hace `toUpperCase()` con un string que ya está en mayúsculas?

Lo deja igual.

16. ¿Por qué se inicializa `max = Number.MIN_SAFE_INTEGER` en vez de `max = 0`?

Porque si todos los datos son negativos, `0` siempre va a ser mayor y nunca se actualiza.

17. ¿Qué pasa si dividís entre 0 en JS?

`1 / 0` da `Infinity`. `0 / 0` da `NaN`. Hay que evitarlo con un `if`.

18. ¿Cómo se ignoran mayúsculas y minúsculas al comparar strings?

Pasando ambos a `.toUpperCase()` (o `.toLowerCase()`) antes de comparar.

19. ¿Qué garantiza `Math.abs(x)`?

Que el resultado sea positivo (valor absoluto).

20. ¿Qué es una variable booleana?

Una variable que solo puede tener `true` o `false`.

De código

21. ¿Qué imprime esto?

```
let a = 5;
console.log(a++);
console.log(a);
console.log(++a);
console.log(a);
```

Respuesta: 5, 6, 7, 7.

22. ¿Qué imprime esto?

```
for (let i = 1; i ≤ 5; i++) {
    if (i % 2 == 0) {
        console.log(i);
    }
}
```

Respuesta: 2, 4.

23. ¿Qué retorna `consecutivas("hoooola")`?

true (las tres "o" del medio).

24. ¿Cuántas veces se ejecuta el `console.log` en este código?

```
for (let i = 0; i < 3; i++) {
    for (let j = 0; j < 4; j++) {
        console.log(i, j);
    }
}
```

Respuesta: 12 veces (3 × 4).

25. ¿Qué imprime esto si el usuario ingresa "Hola"?

```
let s = prompt("Texto");
for (let i = s.length - 1; i ≥ 0; i--) {
    console.log(s.charAt(i));
}
```

Respuesta: a, l, o, H (lo recorre al revés).

Para programar (escribilas en papel y después probalas)

26. Función `contarPalabras(frase)` que cuenta cuántas palabras hay (separadas por un espacio).

27. Función `esPalindromo(palabra)` que retorna true si la palabra se lee igual al derecho que al revés (asumir sin espacios).

28. Función `empiezaYTermina(texto)` que retorna true si el texto empieza y termina con la misma letra (ignorar mayúsculas).

29. Programa que lee 10 números, muestra cuántos son pares, cuántos impares, y el mayor.

30. Programa que lee n y dibuja un triángulo:

```
*
**
***
****
```

31. Función `multiplos(a, b)` que retorna true si a y b son múltiplos entre sí (a divisible por b o b divisible por a).

32. Programa que lee un string y muestra cuántas vocales tiene (todas, sin importar mayúsculas).

33. Función `intercalar(s1, s2)` que recibe dos strings (pueden tener distinto largo) y devuelve uno nuevo con las letras alternadas.

34. Programa que lee n y dibuja un rombo:

```
  *
 ***
*****
 ***
  *
```

35. Función `cantidadVocalesDistintas(texto)` que devuelve cuántas vocales **distintas** aparecen en el texto (entre 0 y 5).

Apéndice: Cheat sheet rápido

¿Qué es un "cheat sheet"?

Cheat sheet (literalmente "hoja de trampa") es una **lista compacta de referencia rápida**. No tiene explicaciones — es solo la sintaxis y los nombres, todos juntos en un lugar para consultar de un vistazo.

Cómo usarlo: No es para aprender desde cero. Es para que, una vez que ya entendiste un concepto leyendo las partes anteriores, puedas tener la sintaxis exacta a mano sin tener que buscar en el documento.

> Estudiá las explicaciones de las partes anteriores. Cuando ya las tengas claras, repasá esta sección como recordatorio rápido el día antes del parcial.

Sintaxis para copiar mental

```javascript
// Variables
let nombre = valor;
const NOMBRE = valor;

// Pedir y mostrar
let x = parseInt(prompt("Mensaje"));
alert("Mensaje");
console.log("Mensaje");

// Condicional
if (cond) {
    // ...
} else if (cond) {
    // ...
} else {
    // ...
}

// While
while (cond) {
    // ...
}

// Do-while
do {
    // ...
} while (cond);

// For
for (let i = 0; i < n; i++) {
    // ...
}

// Función
function nombre(par1, par2) {
    let resultado;
    // ...
    return resultado;
}

// Invocación
let x = nombre(arg1, arg2);
```

Métodos de string

```
texto.length                // largo (sin paréntesis)
texto.charAt(i)             // carácter en posición i
texto.at(i)                 // igual, acepta negativos
texto.toUpperCase()         // todo a mayúsculas
texto.toLowerCase()         // todo a minúsculas
texto.includes(otro)        // ¿contiene otro?
texto.indexOf(otro)         // posición de la primera aparición (-1 si no)
texto.lastIndexOf(otro)     // posición de la última aparición
texto.startsWith(otro)      // ¿empieza con otro?
texto.endsWith(otro)        // ¿termina con otro?
texto.slice(a, b)           // pedazo desde a hasta b (b exclusivo)
texto.substring(a, b)       // similar, acomoda si a > b
texto.trim()                // quita espacios al inicio y final
texto.repeat(n)             // repite n veces
texto.concat(otro)          // concatena (igual que +)
```

Conversiones

```
parseInt("123")             // 123
parseFloat("12.5")          // 12.5
Number("123")               // 123
(25).toString()             // "25"
(9.5467).toFixed(2)         // "9.55"
```

Math

```
Math.abs(-5)                // 5
Math.max(3, 7)              // 7
Math.min(3, 7)              // 3
Math.pow(2, 3)              // 8
Math.sqrt(25)               // 5
Math.PI                     // 3.14159 ...
```

Number

```
Number.MAX_SAFE_INTEGER   // entero positivo más grande
Number.MIN_SAFE_INTEGER   // entero negativo más chico
Number.MAX_VALUE          // número con decimales más grande
Number.MIN_VALUE          // número positivo más cercano a 0
```

Operadores especiales

```
a % b           // resto de a/b
a ** b          // a elevado a b
a += b          // a = a + b
a++             // a = a + 1
&&              // and
||              // or
!               // not
==              // igual (sin tipo)
===             // igual estricto (con tipo)
```

Cierre

El parcial es **hoy, martes 28 de abril**. Tenés todo lo que necesitás.

Recordá lo más importante del último día de clase:

- **Empezá por el ejercicio 1** — vale 7 puntos, es el más fácil.
- **Pensá como robot**, no como humano: leé los ejemplos paso a paso, eso te muestra el algoritmo.
- **No siempre necesitás un for**: a veces hay funciones (`parseInt` , `slice` , `includes`) que resuelven la cosa de una.
- **Hacé corridas a mano** en el borrador con ejemplos chicos antes de pasar a limpio.
- **Punto y coma** al final de cada sentencia.

- **Un solo `return`** por función.
- **`let`, no `var`**.
- **Booleanos**: `if (x)` no `if (x == true)`. `return cond` no `if (cond) return true; return false;`.
- **Nombres claros y en camelCase** (`puntajeMaximo`, no `pm`).
- **Indentación prolija**.
- Si tenés dudas con la **letra**, podés preguntarle al docente.

Suerte. Vos podés.

Documento generado a partir del análisis exhaustivo de las 16 transcripciones de clase (incluyendo la del 27/4/2026, último día antes del parcial) y los parciales anteriores 2024–2025. Material 100% basado en lo dicho por el Prof. Gonzalo Wagner, los profes de práctico Sergio Cortinas (Oliver) y Andrés Mauricio Repeto. Programación 1 — ORT Uruguay, Ingeniería en Sistemas, 1er semestre 2026.